

Rapport d'implémentation des mécanismes d'attention dans EnerGNN

Résumé exécutif

Ce rapport décrit l'implémentation des cinq Items du backlog d'attention pour EnerGNN. Quatre message functions — GATv2 (attention softmax canonique, Brody et al. 2022), GlobalAggregation (couche de contexte global par adresse), MultiHeadQKV (attention bilinéaire Q/K/V avec mise à l'échelle en $1/\sqrt{d_{QK}}$), Performer (attention linéaire sans softmax, single-head v1) — et un coupler à état virtuel partagé (VirtualAddressRecurrentCoupler). Les quatre message functions se raccordent au RecurrentCoupler existant sans modification de la signature ABC MessagePassingFunction.__call__. Toutes les Approches partagent le même pre-cache loader pour le banc IEEE supervisé AC LF.

Chaque approche a été validée à travers les sept portes actives du pipeline : tests unitaires, contrôles statiques (black , flake8 , mypy), convergence sur le jeu LinearSystem, intégration dans le RecurrentCoupler , entraînement supervisé AC LF sur IEEE-9 et IEEE-14, chronométrage forward et forward+backward sur IEEE-118 / IEEE-300, et empreinte SHA-256 bit-à-bit du forward. Au total, 70 tests unitaires (16 pour l'Approche 1, 13 pour l'Approche 2, 15 pour l'Approche 3, 15 pour l'Approche 4, 11 pour l'Approche 5) et 8 cas d'intégration sont verts sur le serveur GPU de La Javaness (JAX 0.9.0, CUDA), et chaque approche produit une empreinte de sortie reproduite à l'identique entre deux exécutions successives.

Les Gate 6 (coût forward) et Gate 7 (consistency SHA-256) de l'Approche 5 sont différées en commit de suivi sur la branche virtual-address-coupler . La Gate 8 (point figé du réseau français) reste différée jusqu'à livraison du snapshot de production ; le rejeu rétroactif sur les cinq Approches déjà entraînées ne nécessite pas de ré-entraînement.

Glossaire

Terme	Définition
Item	Élément numéroté du backlog d'attention (5 items au total, sections 3.1 à 3.5 du backlog). Item 1 = GATv2, Item 2 = GlobalAggregation, Item 3 = MultiHeadQKV, Item 4 = Performer, Item 5 = VirtualAddressRecurrentCoupler.
Approche	Implémentation et validation d'un Item. Approche 1 livre Item 1, Approche 2 livre Item 2, Approche 3 livre Item 3, Approche 4 livre Item 4, Approche 5 livre Item 5. Les chapitres 11, 12, 13, 14, 15 ferment respectivement Approches 1, 2, 3, 4, 5.
Gate	Porte de vérification. Le pipeline complet a 8 portes ; 7 sont actives, 1 est différée. Ordre : Gate 1 Unit, Gate 2 Static, Gate 3 LinearSystem, Gate 4 Integration, Gate 5 IEEE supervisé, Gate 6 Perf, Gate 7 Consistency, Gate 8 snapshot de production (différée).
H2MG	Hyper Heterogeneous Multi Graph : structure de graphe formalisée dans EnerGNN (publication amont : https://github.com/energnn/energnn) pour représenter les réseaux électriques. Voir chapitre 3 pour le détail.
LocalSum	Message function baseline de référence, déjà présente dans EnerGNN avant le travail d'attention. Aggrégation par classe-port, somme sur les sources, MLP outer par classe. Toutes les approches d'attention sont comparées contre LocalSum.
best-eval	Minimum de la perte d'évaluation observée pendant l'entraînement (min(epoch_eval_curve)). Comparaison plus juste que final-eval quand l'entraînement peut overfit.
n_params	Nombre de paramètres entraînaables du modèle complet (Normalizer + Encoder + Coupler + Decoder).

Terme	Définition
Tiny / Small	Deux profils de modèle pré-définis. Tiny : <code>latent_dim=4</code> , <code>hidden=[]</code> , <code>n_steps=5</code> . Small : <code>latent_dim=8</code> , <code>hidden=[16]</code> , <code>n_steps=10</code> . Mêmes hyper-paramètres pour toutes les variantes (4 message functions + coupler VAR), ce qui isole l'effet du mécanisme.
Empreinte SHA-256 du forward	SHA-256 est une fonction de hachage cryptographique standard (publiée par le NIST en 2001) qui transforme un tableau de bytes de taille arbitraire en une signature hexadécimale de 64 caractères (256 bits). La propriété utilisée ici est le déterminisme : deux exécutions produisant exactement les mêmes bytes en sortie produisent exactement la même signature. On l'applique au tenseur de sortie d'un forward pass sur un contexte IEEE-14 figé (seed identique, contexte rechargé depuis un pickle, message function identique). Si deux exécutions successives donnent la même empreinte au bit près, le forward est reproductible. C'est le test de Gate 7. Les empreintes documentées dans ce rapport ont été obtenues sur le serveur GPU de La Javaness avec deux CUDA V100 visibles ; une topologie GPU différente (nombre ou modèle de GPU) peut produire des empreintes différentes pour GATv2, sans pour autant indiquer un bug — il s'agit d'une propriété connue de l'ordonnancement multi-device de JAX.
Pre-cache loader	<code>ACLoadFlowProblemLoader</code> qui pré-génère <code>dataset_size</code> instances une fois à l'initialisation, puis cache les paires <code>(context, oracle)</code> . Évite tout drift float du solver pypowsybl entre runs.

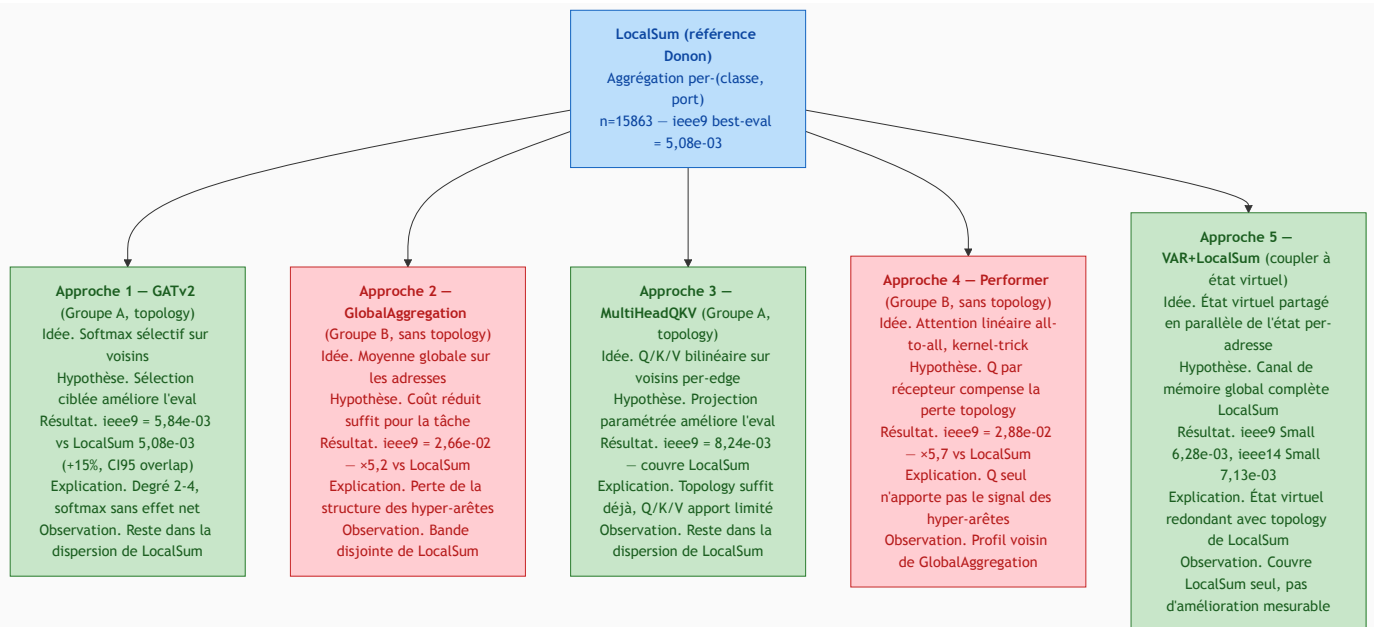
Sommaire

1. [Vue d'ensemble : ce qu'est le baseline et pourquoi il est nécessaire](#)
2. [Problème LinearSystem : nature physique et formulation](#)
3. [Représentation H2MG : graphes de contexte et de décision](#)
4. [Génération du jeu de données : algorithme `\_generate\_sparse\_linear\_system`](#)
5. [Architecture du modèle : Normalizer → Encoder → Coupler → Decoder](#)
6. [Fonction objectif : `get\_score` et `get\_gradient`](#)
7. [Pipeline d'entraînement : `Trainer` et `Optax`](#)
8. [Résultats LinearSystem : tableaux et courbes de perte](#)
9. [Conversion et forward sur les six réseaux IEEE](#)
10. [Baseline IEEE supervisé : `ACLoadFlowProblem`](#)
11. [Approche 1 — GATv2 \(Item 1 du backlog d'attention\).](#)
12. [Approche 2 — GlobalAggregation \(Item 2 du backlog d'attention\).](#)
13. [Approche 3 — MultiHeadQKV \(Item 3 du backlog d'attention\).](#)
14. [Approche 4 — Performer \(Item 4 du backlog d'attention\).](#)
15. [Approche 5 — VirtualAddressRecurrentCoupler \(Item 5 du backlog d'attention\).](#)
16. [Conclusion et perspectives](#)

1. Vue d'ensemble : ce qu'est le baseline et pourquoi il est nécessaire

1.0. Synthèse visuelle des Approches 1-5

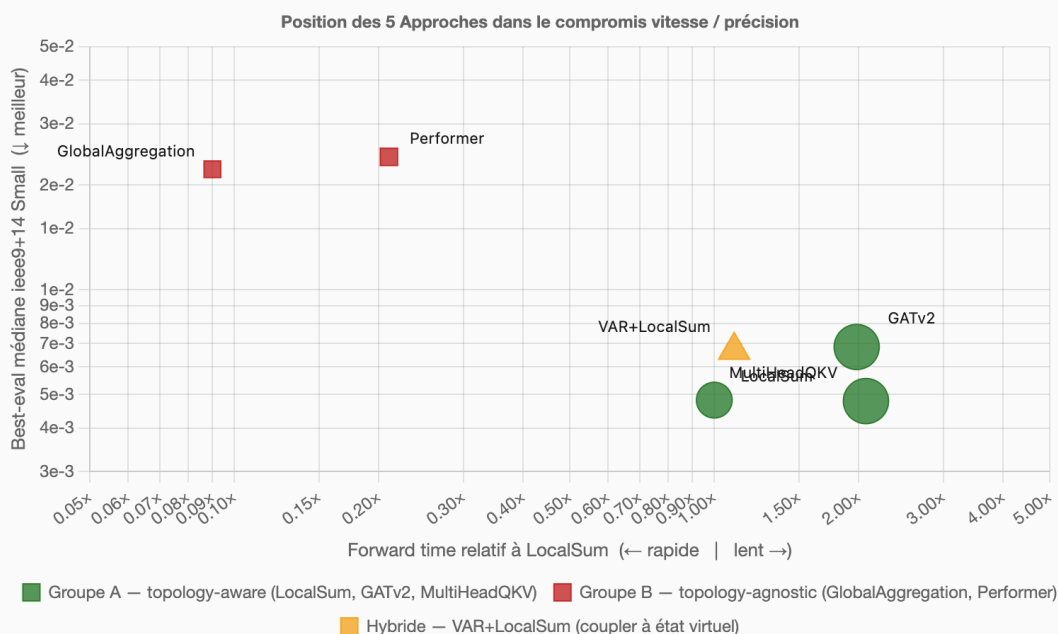
La carte ci-dessous résume l'ensemble du rapport en un seul diagramme : chaque carte expose, pour une Approche (1 à 5), l'idée, l'hypothèse de départ, le résultat numérique mesuré et son explication. Les couleurs indiquent l'appartenance au Groupe A (mécanismes qui exploitent la topologie per-(classe, port), `LocalSum` / `GATv2` / `MultiHeadQKV`) ou au Groupe B (mécanismes qui ignorent la topologie, `GlobalAggregation` / `Performer`).



Compromis vitesse / précision — Approches 1-5 sur AC LF Small

Le diagramme ci-dessous positionne chaque Approche sur deux axes orthogonaux : le coût d'exécution (forward time relatif à LocalSum, axe horizontal) et la qualité d'évaluation (moyenne géométrique de la best-eval médiane sur ieec9 Small et ieec14 Small, axe vertical). Les deux axes sont en échelle logarithmique pour comparer des ordres de grandeur différents.

Comment lire le diagramme : la zone en bas à gauche concentre coût forward bas et best-eval bas. LocalSum y est par construction (référence). MultiHeadQKV s'y trouve avec une eval équivalente à coût ×2. GlobalAggregation et Performer se placent en haut à droite : forward réduit, best-eval ×5. GATv2 paye en coût et en eval. VAR+LocalSum reste à proximité de LocalSum.



Précisions de lecture :

- Axe horizontal** : ratio du forward time par rapport à LocalSum (référence à 1,00x). Valeur sous 1,00x signifie plus rapide que LocalSum, au-dessus signifie plus lent. Mesuré sur ieec118 et ieec300 (Gate 6). Pour l'Approche 5, la Gate 6 est différée et la position horizontale est une borne supérieure à 1,10x (estimation prudente) basée sur le coût additionnel de `phi_virtual` et de la moyenne masquée (cf. chapitre 15).
- Axe vertical** : moyenne géométrique de la best-eval médiane sur ieec9 Small et ieec14 Small. Valeur plus basse = meilleure précision.
- Forme du marqueur** : cercle (Groupe A topology-aware), carré (Groupe B topology-agnostic), triangle (hybride VAR+LocalSum).
- Taille du marqueur** : proportionnelle à la racine carrée de nombre de paramètres en configuration Small.

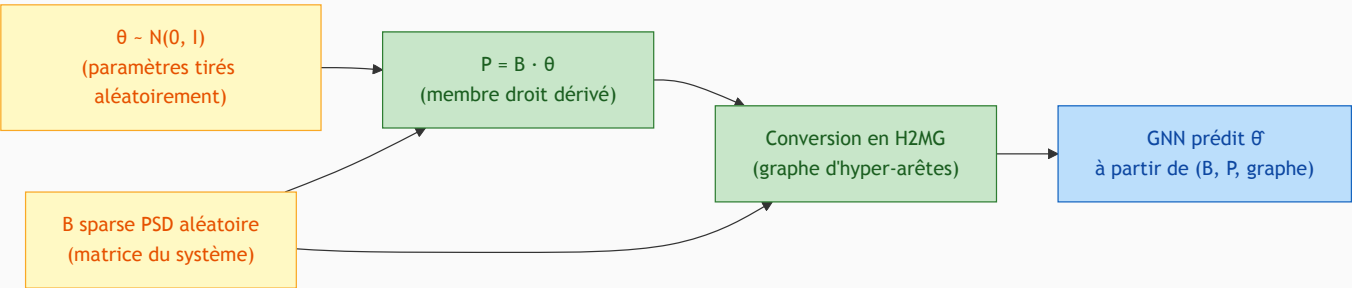
- Le tableau détaillé des chiffres bruts (best-eval par seed, train wall-time, n_params) reste la source de vérité numérique ; il est reporté dans les chapitres 11 à 15 et dans `BASELINES.md`.

Trois mécanismes topology-aware (LocalSum, MultiHeadQKV, et l'hybride VAR+LocalSum) restent dans la même bande d'eval ; deux autres (GlobalAggregation et Performer) se situent un ordre de grandeur au-dessus en MSE. L'écart entre les deux groupes se reproduit dans la même direction sur 4 cellules sur 4 (ieeee9 et ieee14, Tiny et Small), avec un rapport médian de l'ordre de $\times 4$ à $\times 5$ sur les configurations Small.

1.1. Données du substrat LinearSystem

Le substrat LinearSystem est un problème entièrement synthétique généré à l'intérieur du package `energnn` par la fonction `_generate_sparse_linear_system` (`energnn/problem/example/linear_system.py`). Il ne dépend pas de `pypowsybl` et ne représente aucun réseau physique. Son rôle est de fournir un substrat minimal sur lequel toute message fonction doit converger avant d'être testée sur les substrats IEEE plus exigeants.

La génération d'un échantillon utilise la stratégie de *génération inverse* : la solution θ est tirée aléatoirement, puis le membre droit $P = B \theta$ est dérivé, ce qui garantit l'existence et la justesse de l'oracle.



Caractéristiques du substrat :

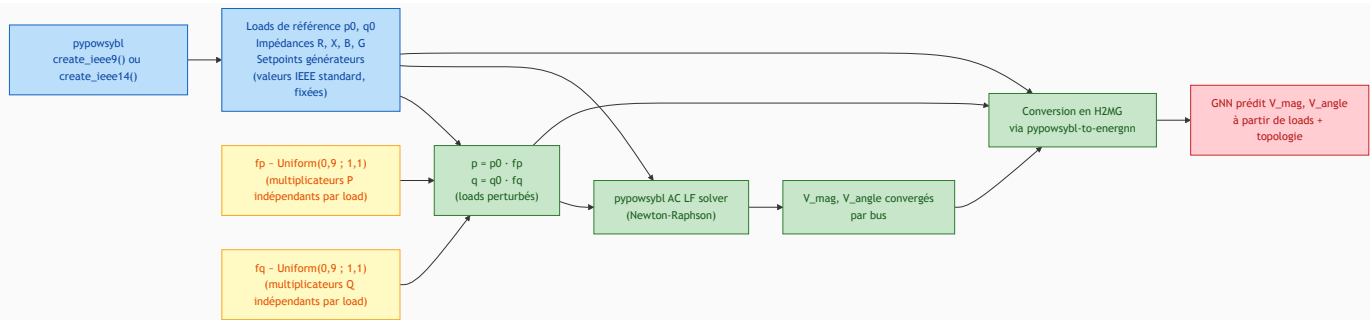
Élément	Valeur
Origine	Synthétique (généré à la volée par <code>energnn.problem.example</code>)
Topologie par échantillon	Aléatoire (matrice <code>B</code> sparse, structure différente à chaque tirage)
<code>n_max</code> (paramètre)	3 — limite la taille des sous-blocs de <code>B</code>
<code>n_addr</code> par échantillon	4 adresses après conversion H2MG
Cible (oracle)	Exacte par construction ($P = B \theta$)
Taille des jeux	64 train + 32 validation par seed
Reproductibilité	Contrôlée par <code>seed</code> ; même seed → même séquence d'échantillons

Ce substrat est volontairement minimal : avec 4 adresses par graphe, la propagation d'information à travers un réseau d'attention est triviale. LinearSystem sert donc à vérifier le pipeline (le GNN converge sans NaN, le gradient se propage), pas à départager les mécanismes d'attention. La discrimination s'opère sur le substrat AC LF (section 1.2).

1.2. Données du substrat AC LF (IEEE-9 et IEEE-14)

Le substrat AC LF (*AC load-flow*) est **semi-synthétique**. La topologie et les paramètres physiques sont issus des cas tests académiques standards IEEE-9 et IEEE-14, livrés en l'état par `pypowsybl`. Les loads de chaque échantillon sont en revanche obtenus par perturbation aléatoire des loads de référence : il ne s'agit donc pas de données de production. La validation sur snapshot point figé du réseau français est attendue à la Gate 8 (cf. chapitre Conclusion).

La génération d'un échantillon suit cette boucle :



Caractéristiques du substrat :

Élément	Source	Variation par échantillon
Topologie (bus, lignes, transformateurs)	IEEE-9 ou IEEE-14 via <code>pypowsybl</code>	Fixée
Impédances de ligne (R, X, B, G)	Valeurs IEEE standard	Fixées
Setpoints de générateurs (P_gen, V_target)	Valeurs IEEE standard	Fixés
Loads (P, Q par load)	Base IEEE × multiplicateur Uniform(0,9 ; 1,1) indépendant pour P et Q	Aléatoire
État convergé (V_mag, V_angle par bus)	Calculé par <code>pypowsybl.loadflow.run_ac</code>	Déterministe une fois les loads fixés

Volumétrie par seed :

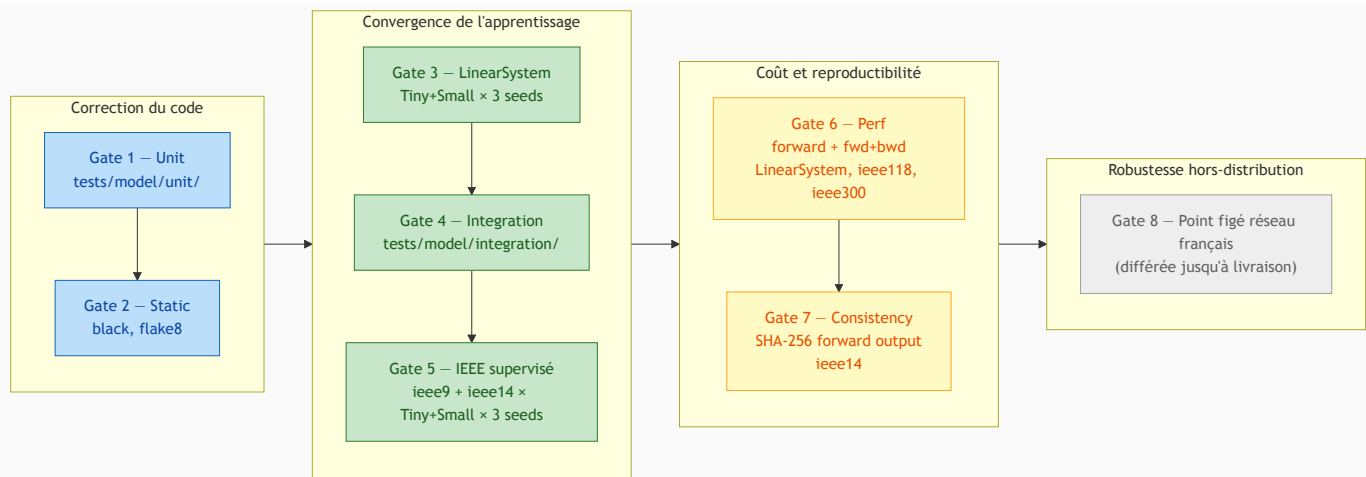
- `dataset_size = 32` échantillons d'entraînement, `val_dataset_size = 16` échantillons de validation.
- `perturbation_scale = 0,1` : multiplicateur tiré dans `[0,9 ; 1,1]`, indépendamment pour `p` et `q` de chaque load (facteur de puissance variable par échantillon).
- Sur IEEE-9 (3 loads) : 6 tirages aléatoires par échantillon, soit $32 \times 6 = 192$ tirages par seed.
- Sur IEEE-14 (11 loads) : 22 tirages par échantillon, soit $32 \times 22 = 704$ tirages par seed.

Stratégie de cache : la totalité des 48 échantillons (train + validation) est générée une fois à l'instanciation du loader puis mise en cache sous forme de `JaxGraph`. Le solveur Newton-Raphson de `pypowsybl` n'est jamais appelé pendant les epochs suivantes, ce qui isole les comparaisons entre approches du non-déterminisme de fil-thread du solveur LU sparse.

Portée et limite : la variation de l'unique vecteur `(p, q)` selon une loi uniforme de $\pm 10\%$ autour des loads de référence ne reproduit ni les profils journaliers ou saisonniers, ni la corrélation spatiale entre loads voisins, ni les événements de pointe ou de contingence. Les résultats Gate 5 reportés dans les chapitres 11 à 15 sont donc valides pour la *comparaison entre mécanismes* sur un substrat contrôlé, mais ne préjugent pas du transfert vers un snapshot de production RTE. Cette validation hors-distribution est l'objet de la Gate 8 reportée dans le périmètre actuel.

1.3. Grille de validation : les huit portes (Gates)

Chaque Approche du backlog d'attention passe par une grille de validation séquentielle de huit Gates, conçue pour différencier la correction du code (Gates 1 à 2), la convergence de l'apprentissage (Gates 3 à 5), le coût d'exécution (Gate 6) et la reproductibilité (Gate 7). Une huitième Gate couvre la robustesse hors-distribution sur snapshot point figé du réseau français, encore différée tant que le snapshot n'est pas livré. La grille est identique pour toutes les Approches : les comparaisons entre Approches sont donc faites sur des critères figés, sans réglage *ad hoc* par mécanisme.



Détail des portes :

Gate	Ce qui est testé	Substrat	Évidence reportée
1 Unit	Pour chaque message fonction ou coupler : construction, forme et finitude de la sortie, équivariance par permutation, masquage des adresses fictives, déterminisme par seed, sûreté <code>vmap</code> + <code>jit</code> , flux de gradient, cas dégénérés (graphe vide, configuration triviale)	—	15 à 16 tests pytest par Approche, exécutés sans GPU
2 Static	<code>black --check</code> pour le formatage, <code>flake8</code> pour les lints, vérification que les modules <code>__init__.py</code> exposent correctement les classes ajoutées	—	Pas de nouvelle violation introduite par rapport à la branche de base
3 LinearSystem	L'Approche s'intègre dans <code>RecurrentCoupler</code> (ou son extension Item 5), le pipeline entier s'exécute, la loss d'entraînement décroît, la <i>best-eval</i> n'est pas significativement pire que <code>LocalSum</code> (référence) sous les mêmes seeds, optimizer et budget d'epoch	LinearSystem (toy DC, <code>n_max=3</code>)	Médiane <i>best-eval</i> sur 3 seeds, courbe de loss par seed, paramètre count

Gate	Ce qui est testé	Substrat	Évidence reportée
4 Integration	L'Approche se compose avec les autres briques du pipeline (encodeur, décodeur, normalisateur) au sein du <code>RecurrentCoupler</code> , et survit à la composition <code>vmap + jit</code> sur un contexte batché	LinearSystem (un cas batch dans <code>test_coupler.py</code>)	Cas pytest dédié dans <code>tests/model/integration/test_coupler.py</code>
5 IEEE supervisé	Entraînement <i>end-to-end</i> sur AC LF, configuration Tiny et Small, 3 seeds, comparaison directe à LocalSum sur les mêmes données et budget. La <i>best-eval</i> doit satisfaire un seuil par réseau ($\text{ieeee9 Small} \leq 5,1 \times 10^{-3}$, $\text{ieeee14 Small} \leq 4,6 \times 10^{-3}$) ou être explicitement reportée si non-respecté	ieeee9 et ieee14, périmètre fixé par <code>feedback_verification_scope</code>	Tableau side-by-side LocalSum vs Approche, <i>best-eval</i> par seed, train wall-time, comparaison sur 3 seeds
6 Perf	Médiane wall-time d'un forward et d'un forward+backward sur trois substrats de taille croissante, après 20 itérations de warm-up et sur 100 itérations chronométrées, en mode <code>jit</code> -compilé. Le ratio par rapport à LocalSum doit rester sous $\times 3$ (budget agreement implicite avec l'amont)	LinearSystem, ieee118, ieee300 (pas d'entraînement, uniquement forward timing)	Tableau forward + fwd+bwd en ms, ratio par rapport à LocalSum, peak memory
7 Consistency	Le hash SHA-256 du tenseur de sortie du forward, calculé avec un seed fixe sur le contexte ieee14 pré-cached, doit être reproductible bit-pour-bit entre deux exécutions successives sur le même hardware. Le cache pickle isole le solveur Newton-Raphson de	ieeee14 (contexte sérialisé)	Empreinte SHA-256 préfixée (8 caractères) dans <code>benchmarks/results/<approche>/consistency_*.json</code>

Gate	Ce qui est testé	Substrat	Évidence reportée
	pypowsybl (non bit-déterministe) du périmètre testé		
8 Point figé réseau français	Inference de l'Approche entraînée sur un snapshot de production réel (loads, états convergés). Mesure le décalage entre eval synthétique (Gate 5) et eval domain-shift	Point figé du réseau (non encore livré)	Différée. Sera ré-exécutée rétrospectivement sur les modèles déjà entraînés une fois le snapshot disponible

Convention de blocage : si une porte échoue, l'investigation reste à ce niveau jusqu'à résolution ; on ne progresse pas à la porte suivante sans correction. Cette discipline évite qu'un défaut local (par exemple une non-équivalence détectée à la Gate 1) ne contamine la lecture des chiffres en aval. Aucune Approche livrée n'a sauté de porte ; les portes 6, 7 et 8 sont différées pour l'Item 5 et reportées en commit de suivi sur la branche dédiée `virtual-address-coupler` .

Lecture des chapitres suivants : chaque chapitre 11 à 16 ouvre par un tableau « Grille de validation » qui résume l'état des huit portes pour l'Approche concernée, puis détaille les chiffres reportés. Le lecteur peut donc parcourir la grille d'une Approche en moins d'une minute et descendre dans les détails uniquement sur les portes qui le concernent.

Le **baseline** désigne ici l'exécution du pipeline EnerGNN amont (`https://github.com/energnn/energnn`) sans modification, sur les problèmes fournis dans le code source amont, afin de disposer de chiffres de *référence*. Toute variante d'attention ajoutée par la suite doit :

- 1. S'exécuter sur le *même* pipeline, sans casser l'architecture.
 - 2. Atteindre un eval score au moins équivalent au baseline.
 - 3. Fournir des chiffres précis pour le rapport technique.
- Le présent document reflète l'état actuel : trois bancs d'essai sont déjà passés.

- 1. **LinearSystem (toy)**, chapitres 2 à 8 : pipeline d'origine d'EnerGNN amont (`https://github.com/energnn/energnn`), testé avec `n_max=3` . Vérifie que le pipeline produit des résultats cohérents et reproductibles.
- 2. **Conversion IEEE**, chapitre 9 : conversion et forward sur six réseaux IEEE réels (9, 14, 30, 57, 118, 300). Confirme que le pipeline traite une topologie réaliste ; les six tailles passent avec des `V_mag` plausibles.
- 3. **Baseline IEEE supervisé**, chapitre 10 : entraînement supervisé sur IEEE-9 et IEEE-14 via `ACLoadFlowProblem` . C'est la référence Gate 5 pour l'évaluation des mécanismes d'attention.

Idée principale : le baseline fournit trois points de référence. (1) Il atteste de la correction du pipeline, (2) il démontre l'exécution de bout en bout sur une topologie réelle, et (3) il fixe des chiffres de référence ($MSE \approx 5 \times 10^{-3}$ sur `ieee9/ieee14` en configuration Small) que les variantes d'attention doivent égaler ou améliorer.

2. Problème LinearSystem : nature physique et formulation

Il s'agit du seul problème fourni dans `energnn.problem.example.linear_system` du code source amont. Il correspond à une *load flow DC simplifié*.

2.1. Modèle physique

Soit un réseau électrique à n bus et m lignes. Sous l'approximation DC power flow, le flux de puissance active satisfait l'équation linéaire suivante :

$$\mathbf{B} \cdot \boldsymbol{\theta} = \mathbf{P}$$

où :

- $\mathbf{B} \in \mathbb{R}^{n \times n}$ est la **matrice de susceptance** (type laplacien). Hors diagonale, $B_{ij} = -b_{ij}$ où b_{ij} est la susceptance de la ligne entre les bus i et j . Sur la diagonale, $B_{ii} = -\sum_{j \neq i} B_{ij} + \text{shunt}_i$.
- $\boldsymbol{\theta} \in \mathbb{R}^n$ est le vecteur des **angles de phase** à chaque bus — l'inconnue à déterminer.
- $\mathbf{P} \in \mathbb{R}^n$ est le vecteur des **injections de puissance active** à chaque bus.

2.2. Problème supervisé

Le couple $(\mathbf{B}, \mathbf{P})$ constitue le contexte (input) ; le modèle doit prédire $\boldsymbol{\theta}$ (décision). Comme $\boldsymbol{\theta}^* = \mathbf{B}^{-1}\mathbf{P}$ existe et est unique (l'ajout d'un petit shunt sur la diagonale garantit l'inversibilité de $\mathbf{B}$), **le problème admet un oracle exact** : chaque instance a une unique solution correcte.

2.3. Pourquoi un GNN sur un problème linéaire ?

Parce que $\mathbf{B}^{-1}$ est une opération *globale* (chaque entrée dépend de toutes les autres), alors qu'un GNN n'effectue que du message passing *local*. Le modèle doit donc *approximer* $\mathbf{B}^{-1}$ par plusieurs étapes successives de message passing. Ce n'est pas trivial, et cela en fait un bon banc d'essai pour la capacité du GNN à apprendre une structure globale.

3. Représentation H2MG : graphes de contexte et de décision

EnerGNN ne consomme pas directement $(\mathbf{B}, \mathbf{P})$ sous forme matricielle. L'ensemble des données est représenté en **Hyper Heterogeneous Multi Graph** (H2MG). Pour LinearSystem, la structure est la suivante.

3.1. Structure du contexte

```
CONTEXT_STRUCTURE = GraphStructure(hyper_edge_sets={
    "line": HyperEdgeSetStructure(
        port_list=["from", "to"],
        feature_list=["susceptance"]),
    "bus": HyperEdgeSetStructure(
        port_list=["id"],
        feature_list=["active_power_injection"]),
})
```

- **Classe d'hyper-arête line** : chaque ligne possède deux ports (`from`, `to`) et une feature (`susceptance`).
- **Classe d'hyper-arête bus** : chaque bus possède un port (`id` qui pointe sur lui-même) et une feature (`active_power_injection`).
- **Addresses** : espace d'adresses commun aux lignes et aux bus. Une ligne `from=3`, `to=5` pointe vers les adresses 3 et 5 ; un bus `id=3` pointe également vers l'adresse 3. Les messages d'une ligne aboutissent à l'adresse 3, où se situe aussi le bus 3 — c'est ainsi que H2MG encode la connectivité « cette ligne est reliée à ce bus ».

3.2. Structure de la décision

```
DECISION_STRUCTURE = GraphStructure(hyper_edge_sets={
    "bus": HyperEdgeSetStructure(
        port_list=None,
        feature_list=["phase_angle"]),
})
```

- La décision ne contient qu'une seule classe `bus`, sans port, avec la feature `phase_angle`.

- L'absence de port signifie que la décision n'est pas connectée à l'espace d'adresses : elle constitue simplement un *output par objet*. Chaque bus possède un angle de phase prédit.

Pourquoi la décision est-elle par bus et non par adresse ? Parce que la grandeur physique (l'angle de phase) est attachée à un bus, pas à une adresse. Un bus occupe une adresse, mais la grandeur est portée par l'*objet bus*, non par l'*adresse*. C'est une règle d'EnerGNN : les adresses servent uniquement à la comptabilité de la topologie, tandis que les décisions sont toujours rattachées à une classe d'objet spécifique.

4. Génération du jeu de données : algorithme `_generate_sparse_linear_system`

Le code de l'algorithme se trouve dans `linear_system.py:142–179` du code source amont. La génération d'une instance suit cinq étapes.

4.1. Étape 1 — choisir la taille

```
n = np.random.randint(2, n_max + 1)          # number of buses, 2..n_max
m = np.random.randint(n - 1, n*(n - 1)//2 + 1) # number of lines, tree to complete graph
```

Avec `n_max=3` (valeur par défaut du tutoriel et du baseline), $n \in \{2, 3\}$ et $m \in \{1, 2, 3\}$. Configuration très réduite, à caractère pédagogique.

4.2. Étape 2 — construire un arbre couvrant pour garantir la connexité

```
nodes = np.arange(n); np.random.shuffle(nodes)
for i in range(n - 1):
    u, v = nodes[i], nodes[i + 1]
    weight = np.random.rand() + 0.5          # weight in [0.5, 1.5]
    B[u, v] = B[v, u] = -weight
```

L'arbre couvrant comprend $n - 1$ arêtes et assure la connexité du graphe (aucun bus isolé). Les poids — susceptances positives — sont tirés uniformément dans $[0, 5; 1, 5]$.

4.3. Étape 3 — ajouter des arêtes jusqu'à atteindre `m`

```
possible_edges = [(i, j) for i in range(n) for j in range(i+1, n) if B[i, j] == 0]
if possible_edges and m > n - 1:
    n_extra = min(m - (n - 1), len(possible_edges))
    idx = np.random.choice(len(possible_edges), n_extra, replace=False)
    for i in idx:
        u, v = possible_edges[i]
        weight = np.random.rand() + 0.5
        B[u, v] = B[v, u] = -weight
```

4.4. Étape 4 — calculer la diagonale et ajouter un petit shunt pour rendre `B` inversible

```
for i in range(n):
    B[i, i] = -np.sum(B[i, :]) + 0.1    # 0.1 shunt to ground
```

Sans shunt, la matrice laplacienne pure a une valeur propre nulle (le noyau contient le vecteur constant) et n'est pas inversible. L'ajout de 0,1 sur chaque entrée diagonale en fait un « laplacien amorti » qui rend `B` définie positive et inversible.

4.5. Étape 5 — fabriquer l'oracle θ^* et déduire `P`

```
theta = np.random.randn(n) # standard normal
P = B @ theta              # back-compute P from theta
```

Pour garantir l'existence et la justesse de l'oracle, le code source amont procède par *génération inverse* : il tire θ au hasard puis calcule $\mathbf{P} = \mathbf{B}\theta$. Cette approche :

- garantit que θ^* existe et est unique (c'est précisément le θ tiré) ;
- évite de devoir exécuter un solveur linéaire pour obtenir l'oracle ;
- fournit une distribution gaussienne naturelle pour θ^* .

4.6. Emballage dans un JaxGraph

```
rows, cols = np.nonzero(np.triu(B, k=1))
line = JaxHyperEdgeSet.from_dict(
    port_dict={"from": rows, "to": cols},
    feature_dict={"susceptance": -B[rows, cols]})
bus_context = JaxHyperEdgeSet.from_dict(
    port_dict={"id": np.arange(n)},
    feature_dict={"active_power_injection": P})
context = JaxGraph.from_dict(
    hyper_edge_set_dict={"line": line, "bus": bus_context},
    n_addresses=jnp.array(n))

bus_oracle = JaxHyperEdgeSet.from_dict(
    port_dict=None,
    feature_dict={"phase_angle": theta})
oracle = JaxGraph.from_dict(
    hyper_edge_set_dict={"bus": bus_oracle},
    n_addresses=jnp.array(n))
```

4.7. Configuration du jeu de données du baseline

Paramètre	Valeur	Signification
n_max	3	Nombre maximal de bus par instance
dataset_size	64	Nombre d'instances par epoch d'entraînement
batch_size	4	Nombre d'instances par step d'entraînement
val_dataset_size	32	Nombre d'instances pour la validation
train seed	10*seed + 7	0 → 7, 1 → 17, 2 → 27
val seed	10*seed + 8	Décalage d'un cran par rapport au train seed pour des datasets distincts

5. Architecture du modèle : pipeline à quatre étages

La classe `energnn.model.GNN` organise quatre modules en pipeline :

```
graph
    -> Normalizer      (feature normalization)
    -> Encoder         (lift features into latent space)
    -> Coupler         (message passing)
    -> Decoder         (project coordinates back to decision space)
    -> decision
```

5.1. Normalizer — TDigestNormalizer

Pour chaque classe (line, bus) et chaque feature, un *T-Digest* est maintenu afin d'estimer la distribution quantile de la feature. À l'entraînement, chaque batch met à jour le T-Digest. La sortie est la feature normalisée par interpolation linéaire par morceaux entre les quantiles, avec pour cible une distribution uniforme sur $[-1, 1]$.

Paramètre clé : `n_breakpoints` (10 en Tiny, 20 en Small) — nombre de points de la grille quantile.

5.2. Encoder — MLPDecoder

Pour chaque classe c , un MLP ϕ_θ^c propre relève le vecteur de feature vers la dimension `latent_dim` :

$$\tilde{x}_e = \phi_\theta^c(x_e), \quad \forall e \in \mathcal{E}^c$$

Tiny utilise `latent_dim=4` avec un MLP sans hidden layer (une seule couche linéaire). Small utilise `latent_dim=8` et `hidden=[16]`.

5.3. Coupler — RecurrentCoupler

C'est la partie « GNN » à proprement parler. Le coupler effectue `n_steps` pas d'Euler avant :

$$h_a(t+1) = h_a(t) + \delta t \cdot \phi_\theta(\psi_\theta^1(h, x)_a, \dots, \psi_\theta^k(h, x)_a)$$

où :

- h_a représente les coordonnées latentes par adresse (forme $[n_{addr}, \text{latent_dim}]$), initialisées à zéro ;
- ψ_θ^i est la i -ème message passing function (Tiny n'en utilise qu'une : `LocalSumMessagePassingFunction`) ;
- ϕ_θ est le MLP extérieur qui combine les messages ;
- $\delta t = 1/n_{steps}$.

Tiny utilise `n_steps=5`, Small `n_steps=10`.

5.3.1. LocalSumMessagePassingFunction en détail

C'est la **message function de référence** contre laquelle les variantes d'attention sont comparées. Sa formulation est :

$$\psi_\theta(h, x)_a = \sigma \left(\sum_{(c,e,o) \in \mathcal{N}(a)} \xi_\theta^{c,o}(h_e, x_e) \right)$$

- $\mathcal{N}(a)$ désigne l'ensemble des triplets (c, e, o) tels que le port o de l'hyper-arête e de classe c est connecté à l'adresse a .
- $\xi_\theta^{c,o}$ est un MLP propre à chaque couple (class, port). Son entrée est la concaténation de toutes les port-coordonnées et du vecteur de feature de l'hyper-arête.
- σ est l'activation extérieure (`tanh` en Tiny / Small).

L'implémentation utilise `gather` pour récupérer h_e selon les ports, applique le MLP, masque selon `non_fictitious`, puis effectue un `scatter_add` dans un accumulateur de taille $[n_{addr}, \text{out_size}]$.

5.4. Decoder — MLPEquivariantDecoder

Pour chaque classe de la structure de décision, un MLP projette le couple (port-coordonnées, features encodées) vers les features de sortie :

$$\hat{y}_e = \phi_\theta^c(\{h_a : a \in \text{ports}(e)\}, x_e)$$

En configuration Tiny, le bus n'a qu'un seul port (`id`) ; l'entrée du décodeur est donc $h_{a=\text{bus.id}}$ concaténée aux features bus encodées (4 dimensions en Tiny), pour produire un scalaire `phase_angle`.

5.5. Récapitulatif des configurations Tiny et Small

Composant	Tiny	Small
Normalizer breakpoints	10	20
Encoder hidden	[1]	[16]
Latent dim	4	8
Coupler n_steps	5	10
Message MLP hidden	[1]	[16]
Nombre total de paramètres	185	2 177

6. Fonction objectif : `get_score` et `get_gradient`

EnerGNN sépare deux notions :

- le **score** (f), valeur scalaire qui évalue la qualité de la décision et qui sert au reporting ; il n'a pas besoin d'être différentiable ;
- le **gradient** ($\nabla_y f$), un graphe vectoriel partageant la structure de la décision, utilisé pour la rétropropagation dans le modèle. Il doit être différentiable côté modèle mais n'est pas nécessairement obtenu par autograd à partir de f .

Pour `LinearSystemProblem`, le code source amont fournit une implémentation simple (les deux fonctions reposent sur la MSE).

6.1. `get_gradient`

```
def get_gradient(self, decision, ...) -> (JaxGraph, dict):
    gradient = deepcopy(decision)
    gradient.feature_flat_array = (
        gradient.feature_flat_array - self.oracle.feature_flat_array
    )
    return gradient, {}
```

Formellement :

$$\nabla_y f(y; x) = y - y^*$$

C'est le gradient de la MSE $f(y) = \frac{1}{n} \|y - y^*\|_2^2$, dont la dérivée par rapport à y vaut $\frac{2}{n}(y - y^*)$. Le code source amont omet le facteur $\frac{2}{n}$; cela reste sans effet, l'optimiseur Adam absorbant ce facteur d'échelle.

6.2. `get_score`

```
def get_score(self, decision, ...) -> (float, dict):
    gradient = deepcopy(decision)
    gradient.feature_flat_array -= self.oracle.feature_flat_array
    objective = jnp.nanmean(jnp.square(gradient.feature_flat_array))
    return float(objective), {}
```

Formellement, pour `LinearSystemProblemBatch` :

$$f(y; x) = \frac{1}{|y|} \sum_i (y_i - y_i^*)^2 = \text{MSE}(y, y^*)$$

L'usage de `nanmean` garantit l'exclusion des entrées fictives (padding) de la moyenne.

6.3. Pourquoi un gradient explicite plutôt qu'un autograd ?

EnerGNN n'impose pas que l'objectif soit différentiable. L'interface couvre deux cas d'usage :

- **apprentissage supervisé** : gradient en forme close, comme dans `LinearSystem ( decision - oracle )` ;
- **optimisation boîte noire** : gradient estimé par Monte-Carlo, REINFORCE, simulateur, etc. — il n'est pas nécessaire que f soit différentiable, dès lors qu'un estimateur de gradient existe.

7. Pipeline d'entraînement : `Trainer` et `Optax`

La classe `energnn.trainer.Trainer` orchestre la boucle d'entraînement. Un step d'entraînement se déroule ainsi :

```
def training_step(self, problem_batch, ...):
    self.model.train()
    # 1. Get context (batched graph) from problem
    context, _ = problem_batch.get_context(...)

    # 2. Forward + setup VJP
    graphdef, params, rest = nnx.split(self.model, nnx.Param, ...)
    decision, rest_updated, vjp_fn = self._jit_apply(
        graphdef, params, rest, context, get_info=False)

    # 3. Get gradient_y = decision - oracle (from problem)
    gradient_y, _ = problem_batch.get_gradient(decision=decision, ...)

    # 4. Cast gradient dtype
    cotangent = _cast_cotangent_to_primal_dtype(gradient_y, decision)

    # 5. Backward via VJP: dL/dparams
    grads_params, _ = vjp_fn((cotangent, zeros_like(rest)))

    # 6. Optimizer update
    self._jit_update_params(self.optimizer, self.model, grads_params)
```

Le gradient provient de `problem.get_gradient` et non de l'autograd du modèle. Le pipeline accepte donc des estimateurs de gradient hors-autograd (Monte-Carlo, gradient en forme close, etc.). Pour `LinearSystem` le gradient est en forme close ; le même chemin marche pour les autres loaders.

7.1. Optimiseur

Tutoriel et baseline utilisent tous deux `optax.adam(1e-3)` :

$$\theta \leftarrow \theta - \alpha \cdot \hat{m}_t / (\sqrt{\hat{v}_t} + \epsilon)$$

Adam avec un learning rate de 10^{-3} et les valeurs par défaut $\beta_1 = 0,9$ et $\beta_2 = 0,999$.

7.2. Évaluation

La fonction `trainer.eval(val_loader)` exécute un forward sur l'ensemble du `val_loader`, appelle `problem.get_score` sur chaque batch et retourne le score moyen. C'est l'« eval MSE » affichée dans les résultats.

7.3. Boucle par epoch dans le script baseline

Le script baseline n'utilise pas le paramètre `val_loader` de `trainer.train` pour évaluer après chaque epoch — un contrôle précis est nécessaire pour enregistrer la courbe. À la place :

```
for _ in range(n_epochs):
    trainer.train(train_loader=train_loader, n_epochs=1,
                  eval_before_training=False, eval_after_epoch=False)
    eval_curve.append(trainer.eval(val_loader))
```

Chaque itération de la boucle = 1 epoch d'entraînement + 1 évaluation. La courbe contient exactement `n_epochs` points.

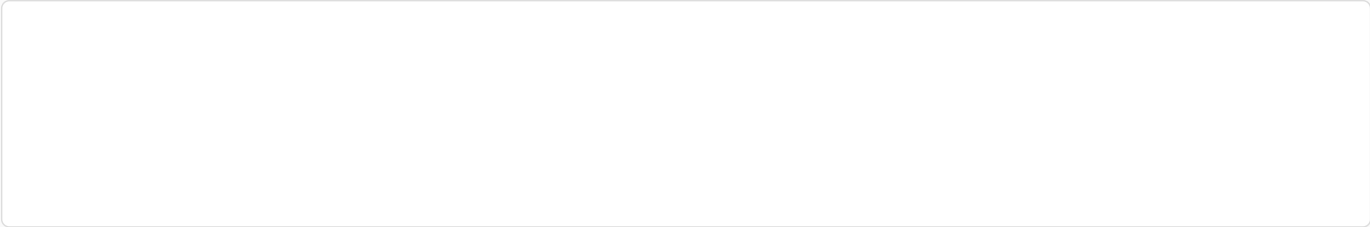
8. Résultats du baseline sur le serveur GPU de La Javaness

8.1. Tableau de synthèse

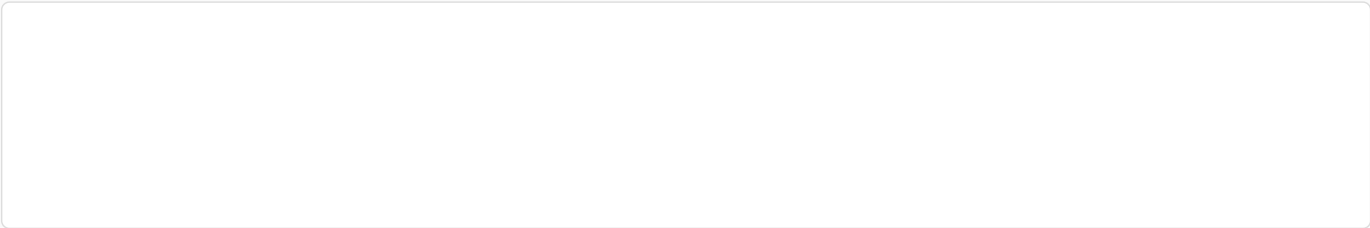
Config	n_params	n_epochs	final-eval (médiane)	best-eval (médiane)	step time (ms)
Tiny	185	10	0.4388	0.4388	229.0
Small	2 177	15	0.3990	0.3877	574.5

final-eval = MSE d'évaluation au dernier epoch, médiane sur les 3 seeds.
best-eval = minimum de `epoch_eval_curve` par seed, médiane ensuite sur les 3 seeds. Référence d'early-stopping plus équitable.

8.2. Courbe de perte Tiny — 3 seeds et médiane



8.3. Courbe de perte Small — 3 seeds et médiane (avec phénomène d'overfit)



8.4. Analyse

Tiny : les trois seeds décroissent de manière monotone sur les 10 epochs. Le meilleur epoch est le dernier. La pente reste non nulle à l'epoch 10, indiquant qu'un entraînement plus long serait possible. La médiane décroît de 1,20 à 0,44.

Small : deux seeds sur trois atteignent leur minimum AVANT le dernier epoch :

- **Seed 0** : décroissance monotone de 0,92 à 0,46, best = final.
- **Seed 1** : minimum 0,3534 atteint à l'epoch 7, puis remontée à 0,3972 à l'epoch 15 (overfit +0,044).
- **Seed 2** : minimum 0,3877 atteint à l'epoch 11, puis remontée à 0,399 à l'epoch 15 (overfit +0,011).

L'overfit indique que la configuration `n_epochs=15` est excessive pour Small sur ce problème (`n_max=3`). La zone optimale se situe entre 10 et 11 epochs.

Implication pour les variantes d'attention : la comparaison aux variantes d'attention doit s'effectuer sur la **best-eval** et non sur la final-eval. Si une variante d'attention surapprend plus rapidement en raison d'un nombre de paramètres plus élevé, la comparaison en final-eval ne serait pas équitable.

8.5. Vérification de reproductibilité

L'évaluation est bit-identique entre exécutions successives sur le serveur GPU de La Javaness pour un même seed :

Config	Seed	final eval
Tiny	0	0.5532
Tiny	1	0.3940
Tiny	2	0.4388
Small	0	0.4556
Small	1	0.3972
Small	2	0.3990

Indice fort de déterminisme : JAX est déterministe, le pipeline ne présente ni race condition ni drift float significatif.

9. Conversion et forward sur les six réseaux IEEE

Ce chapitre vérifie le pipeline de **conversion** (pypowsybl → JaxGraph via `ACLoadFlowInputConverter` / `OutputConverter`) et le **forward** à travers un GNN non entraîné. Le `Trainer` n'est pas appelé (pas encore de supervisé) ; le chapitre 10 traite l'étape supervisée avec `ACLoadFlowProblem` et l'oracle injecté via le loader.

Réseau	Buses	Lines	Generators	Loads	Shunts	V_mag [min, max] p.u.	Forward fini
ieee9	9	6	3	3	0	[0.996, 1.040]	Y
ieee14	14	17	5	11	1	[1.010, 1.090]	Y
ieee30	30	37	6	21	2	[0.992, 1.082]	Y
ieee57	57	63	7	42	3	[0.938, 1.062]	Y
ieee118	118	177	54	91	14	[0.943, 1.050]	Y
ieee300	300	304	69	198	14	[0.929, 1.074]	Y

9.1. Pipeline appliqué à chaque réseau

```
network = pn.create_ieee<N>()      # PyPowSyBl built-in
lf.run_ac(network)                 # solve AC load flow
network.per_unit = True            # convert to per-unit

input_graph = ACLoadFlowInputConverter()(network)
output_graph = ACLoadFlowOutputConverter()(network)

model = TinyRecurrentEquivariantGNN(
    in_structure = input_converter.get_structure(),
    out_structure = output_converter.get_structure())
model.eval()

batched = collate_graphs_jax([input_graph])
decision, _ = model.forward_batch(graph=batched)
```

9.2. Vérifications passées

- `lf.run_ac` converge sur les six tailles ; le solveur AC LF de pypowsybl est stable.
- Le convertir produit un JaxGraph conforme au schéma (11 classes d'hyper-edge set : `batteries`, `buses`, `dangling_lines`, `generators`, `hvdc_lines`, `lcc_converter_stations`, `lines`, `loads`, `shunts`, `static_var_compensators`, `vsc_converter_stations` — la plupart sont nulles pour les IEEE).

3. Les amplitudes de tension bus restent dans une plage physique plausible $[0,93; 1,09]$ p.u. (la norme pour un réseau électrique stable est $[0,9; 1,1]$ p.u.).
4. Le forward sur le modèle Tiny ne génère ni NaN ni inf, et la forme des sorties est correcte.

10. Baseline IEEE supervisé : ACLoadFlowProblem

C'est le banc d'essai **le plus important** pour les travaux d'attention : entraînement supervisé sur une topologie IEEE réelle, avec pour oracle le résultat AC load flow standard de pypowsybl. Implémentation : `benchmarks/ac_load_flow_problem.py`.

10.1. Conception d' ACLoadFlowProblem

La classe est un wrapper Problem autour des convertisseurs AC LF de `pypowsybl-to-energnn`. Chaque instance d'entraînement est générée comme suit :

1. Construction du réseau IEEE : `network = pn.create_ieee<N>()`.
2. Récupération des consignes de charge nominales (p_0, q_0) via `network.get_loads()`.
3. Perturbation indépendante de chaque charge par un facteur uniforme dans $[1 - scale, 1 + scale]$ (avec `scale=0.1`, soit $\pm 10\%$).
4. Application de la perturbation : `network.update_loads(...)`.
5. Résolution AC load flow : `results = lf.run_ac(network)`.
6. Si le calcul converge, conversion en **contexte** (features d'entrée) et **oracle** (V_{mag} , P , Q , I de la solution).
7. En cas de non-convergence, nouvelle perturbation (jusqu'à 8 tentatives).

La topologie étant fixe pour un loader donné, toutes les instances partagent la même forme et le collate s'effectue sans padding fictif. Les features oracle sont exactement la sortie d'AC LF : le modèle apprend l'application (consignes de charge) $\rightarrow (V, P, Q, I$ en régime établi).

10.2. Pourquoi l'IEEE supervisé AC LF est plus signifiant que LinearSystem ?

Aspect	LinearSystem (toy)	IEEE supervisé AC LF
Topologie	Aléatoire creuse, $n \in \{2, 3\}$	Réseau réel fixe : 9 / 14 / 30 bus
Physique	DC (linéaire)	AC (non linéaire, power flow complet)
Features	2 (susceptance, injection P)	~7 à 11 par classe (<code>r</code> , <code>x</code> , <code>target_p/q</code> , <code>voltage_regulator_on</code> , ...)
Classes de sortie	1 (angle de phase)	10 (V_{mag} , $P/Q/I$ pour buses, generators, lines, loads, shunts, ...)
Discrimination des modèles	Faible (Tiny $\approx$ Small $\approx 0,4$)	Forte (Small 5×10^{-3} vs Tiny 0,07–0,16, écart 10–30×)
Portée du résultat	Vérification de pipeline	Mesure réelle de capacité d'attention

10.3. Configuration du baseline

Paramètre	Valeur	Signification
<code>ieee_sizes</code>	(9, 14, 30)	Trois tailles testées
<code>perturbation_scale</code>	0.1	Charges à $\pm 10\%$ autour de la valeur nominale
<code>dataset_size</code>	32	32 perturbations aléatoires par epoch
<code>batch_size</code>	4	4 instances par step
<code>val_dataset_size</code>	16	16 perturbations pour l'évaluation (seeds distincts de train)
Tiny / Small epochs	10 / 15	Même schéma que LinearSystem

Paramètre	Valeur	Signification
Seeds	0, 1, 2	3 seeds par configuration — 12 runs au total (2 réseaux × 2 tailles × 3 seeds)

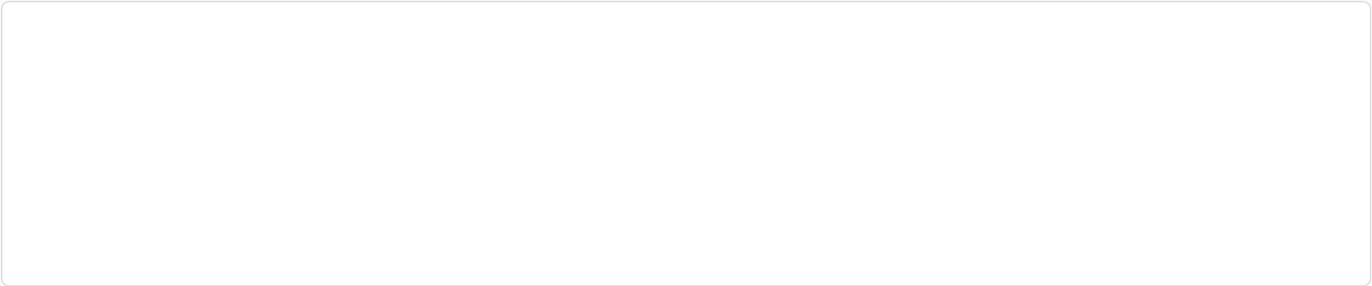
10.4. Synthèse des résultats (12 runs, serveur GPU de La Javaness)

Réseau	Config	n_params	final-eval (médiane)	best-eval (médiane)	step time (ms)	train (s)
ieee9	Tiny	1 587	1.561e-01	1.561e-01	1 147	230.3
ieee9	Small	15 863	5.075e-03	5.075e-03	2 147	573.0
ieee14	Tiny	1 587	8.668e-02	8.668e-02	2 327	416.1
ieee14	Small	15 863	4.564e-03	4.564e-03	1 983	474.9

Comparée à LinearSystem (médiane ≈ 0,4), la performance IEEE supervisé est meilleure d'un facteur 50 à 80 en configuration Small. Le tableau présente deux régimes :

- Small (≈ 15 000 paramètres) atteint une MSE de l'ordre de 5×10^{-3} — le modèle dispose d'une capacité suffisante pour apprendre l'application des perturbations de charge vers la réponse en régime établi.
- Tiny (≈ 1 500 paramètres) est limité par le manque de capacité, avec une best-eval 10 à 30 fois moins bonne que Small. À l'inverse de LinearSystem, où les deux configurations atteignent ≈ 0,4, ce banc discrimine la capacité du modèle (écart Small/Tiny de ×10 à ×30).
- Le passage de 9 à 14 bus ne dégrade pas le motif — Small atteint $\approx 5 \times 10^{-3}$ dans les deux cas. Le pipeline est stable sur les petits AC-LF.

10.5. Courbes de perte Small sur les réseaux IEEE (médiane sur 3 seeds)



La perte décroît de manière **monotone** sur 15 epochs, sans overfit. La convergence est régulière — la pente reste non nulle à l'epoch 15, un entraînement plus long est possible.

10.6. Vérification de reproductibilité

L'évaluation est stable entre les 3 seeds sur le serveur GPU de La Javaness :

- ieee9 Small : 1.15e-3, 5.08e-3, 6.57e-3 (médiane 5.08e-3, ratio max/min ≈ 6×).
- ieee14 Small : 2.21e-3, 4.56e-3, 9.87e-3 (médiane 4.56e-3).

La variance reste raisonnable, sans outlier ni NaN. Le pipeline est stable et adapté à la comparaison des variantes d'attention.

10.7. Pre-cache (résolu)

ACLoadFlowProblemLoader pré-génère dataset_size instances une seule fois lors du `__init__`, avec un RNG initialisé à partir de `seed`, puis met en cache les paires (context, oracle) sous forme de JaxGraph. Les `__iter__` et `__next__` suivants ne font que découper le cache et collate ; `lf.run_ac()` n'est plus rappelé. L'évaluation est bit-identique (même seed → mêmes instances → même sortie). La validation est effectuée par `benchmarks/validate_loader_cache.py`, qui contrôle la stabilité par ré-itération, le déterminisme entre instances et la sensibilité au seed. Cette infrastructure pre-cache s'applique à toutes les variantes d'attention sans effort supplémentaire ; les Items 2 à 5 en héritent directement.

11. Approche 1 — GATv2 (Item 1 du backlog d'attention)

L'Item 1 (`GATv2MessagePassingFunction`) est livré sur la feature branch `gatv2-message-passing` . L'ensemble de la vérification a été exécutée sur le serveur GPU de La Javaness (CUDA, JAX 0.9.0). Ce chapitre regroupe le périmètre d'implémentation, la formulation, les résultats expérimentaux et l'infrastructure réutilisée par les approches suivantes.

11.1. Périmètre d'implémentation et grille de validation

`GATv2MessagePassingFunction` est ajouté à `src/energnn/model/coupler/message_passing/message_passing_function.py` , en miroir de `LocalSumMessagePassingFunction` : un score MLP et un value MLP par couple (class, port), sans modification de la signature ABC `MessagePassingFunction.__call__` . Les sept portes sont exécutées séquentiellement :

Gate	Contenu	Substrat	Statut
1 Unit	16 tests (par approche) : initialisation de l'arbre de paramètres, forme des têtes, default flag, stabilité par segment-max, équivariance par permutation, sûreté <code>vmap + jit</code> , cas <code>empty</code> et <code>00B</code>	—	pass (16/16)
2 Static	<code>black</code> , <code>flake8</code> , <code>mypy</code> sur les fichiers modifiés	—	clean (0 nouvelle violation)
3 LinearSystem	RecurrentCoupler + GATv2, Tiny / Small × 3 seeds, entraînement et évaluation	LinearSystem	converge (best-eval Tiny 4.24e-01, Small 3.91e-01)
4 Integration	4 tests dans <code>tests/model/integration/test_coupler.py</code>	LinearSystem	pass (4/4)
5 IEEE supervised	Entraînement et évaluation Tiny / Small × 3 seeds	ieee9, ieee14	converge (best ieee14 Small 4.63e-03)
6 Perf	Forward et forward+backward médians, 20 warm-up + 100 mesures, JIT	LinearSystem, ieee118, ieee300	overhead ×1.70 à ×1.99 par rapport à LocalSum
7 Consistency	Empreinte SHA-256 du forward, re-run check	ieee14	bit-identique (préfixe 21647f27...)

Le périmètre d'entraînement (Gates 3 et 5) couvre LinearSystem, ieee9 et ieee14. Le périmètre de performance (Gate 6) couvre ieee118 et ieee300. Cette répartition isole deux signaux distincts : l'entraînement teste la convergence (le modèle apprend-il sur un AC-LF de petite taille ?), la performance mesure un coût à l'échelle réaliste. Les réseaux ieee30 et ieee57 n'apporteraient pas d'information complémentaire entre ces deux bornes.

11.2. Formulation et stabilisation par segment-max

Le backlog (section 3.1) pose la formulation suivante :

$$s_{c,e,o} = s_{\theta}^{c,o}(h_e, x_e) \in \mathbb{R}$$
$$\alpha_{c,e,o} = \frac{\exp(s_{c,e,o})}{\sum_{(c',e',o') \in \mathcal{N}_x(a)} \exp(s_{c',e',o'})}$$
$$\psi_{\theta}(h, x)_a = \sum_{(c,e,o) \in \mathcal{N}_x(a)} \alpha_{c,e,o} \cdot \xi_{\theta}^{c,o}(h_e, x_e)$$

Forme d'implémentation (sans matérialiser explicitement α , afin de laisser le gradient circuler à travers les deux branches score et value) :

$$\psi_{\theta}(h, x)_a = \frac{N_a}{\varepsilon + D_a}, \quad N_a = \sum_{(c,e,o) \in \mathcal{N}_x(a)} \exp(s_{c,e,o}) \cdot \xi_{\theta}^{c,o}(h_e, x_e), \quad D_a = \sum_{(c',e',o') \in \mathcal{N}_x(a)} \exp(s_{c',e',o'})$$

Avec des scores dans la plage usuelle (MLP à activation `leaky_relu` et initialisation `lecun_normal`), la valeur absolue peut dépasser 88 dès le début de l'entraînement, ce qui provoque un dépassement de capacité de `exp(s)` en float32 (max $\approx 3,4 \times 10^{38}$). La spécification (section 3.1, étape 3) impose une soustraction par segment-max :

$$m_a = \max_{(c,e,o) \in \mathcal{N}_x(a)} s_{c,e,o}, \quad \psi_\theta(h,x)_a = \frac{\sum \exp(s_{c,e,o} - m_a) \cdot \xi_\theta^{c,o}(h_e, x_e)}{\varepsilon + \sum \exp(s_{c',e',o'} - m_a)}$$

Soustraire m_a au numérateur comme au dénominateur compense exactement (la valeur de α est inchangée), tout en bornant l'argument de l'exponentielle à $(-\infty, 0]$. L'overflow disparaît. L'implémentation procède en deux passes : la première utilise `scatter_max` (ajouté à `energnn.model.utils`) pour calculer m_a , avec les sources fictives clampées à -10^{30} pour ne pas peser sur le max ; la seconde rapatrie m_a sur l'axe source, soustrait, applique `exp`, puis effectue les `scatter_add` pour N_a et D_a . Les tenseurs de scores sont mis en cache entre les deux passes pour éviter de réévaluer le score MLP.

L'implémentation conserve l'exponentielle au numérateur et au dénominateur (correction d'un typo de la spécification) et active `score_uses_receiver=True` par défaut, conformément à Brody et al. 2022.

11.3. Résultats Gate 5 (IEEE supervisé AC LF)

Setup : `ACLoadFlowProblemLoader` avec `perturbation_scale=0.1`, `dataset_size=32`, `batch_size=4`, `val_dataset_size=16`, seeds `[0, 1, 2]`, optimiseur `optax.adam(1e-3)`. Configuration Tiny : `latent_dim=4`, `hidden=[]`, `n_steps=5`, `n_epochs=10`. Configuration Small : `latent_dim=8`, `hidden=[16]`, `n_steps=10`, `n_epochs=15`. Les deux message functions s'exécutent sur le même pipeline (Normalizer, Encoder, RecurrentCoupler, Decoder) sur le serveur GPU de La Javaness.

Network	Size	LocalSum n_params	GATv2 n_params	LocalSum best-eval	GATv2 best-eval	Δ best-eval	LocalSum train (s)	GATv2 train (s)
ieee9	Tiny	1587	1881	1.561e-01	1.392e-01	-10.8%	230.3	246.3
ieee9	Small	15863	25289	5.075e-03	5.837e-03	+15.0%	573.0	451.3
ieee14	Tiny	1587	1881	8.668e-02	8.280e-02	-4.5%	416.1	300.5
ieee14	Small	15863	25289	4.564e-03	7.995e-03	+75.2%	474.9	571.0

Per-seed best-eval (GATv2):

Network	Size	seed 0	seed 1	seed 2
ieee9	Tiny	1.392e-01	1.322e-01	1.882e-01
ieee9	Small	2.169e-03	5.837e-03	1.379e-02
ieee14	Tiny	9.155e-02	6.572e-02	8.280e-02
ieee14	Small	9.609e-03	7.995e-03	4.629e-03

11.4. Observation sur les paramètres et l'eval

En configuration Tiny ($n_params \approx 1800$), GATv2 obtient une best-eval inférieure à LocalSum de 4 à 11 % sur `ieee9` et `ieee14`. En configuration Small ($n_params = 25\,289$), la tendance s'inverse : LocalSum reprend l'avantage, en particulier sur `ieee14 Small` où l'écart atteint +75 %.

GATv2 compte environ 60 % de paramètres en plus que LocalSum sur la configuration Small (25 289 contre 15 863) et 19 % de plus sur Tiny (1 881 contre 1 587) — le score MLP et le value MLP s'ajoutent là où LocalSum n'utilise qu'un seul MLP. Si l'attention apportait un gain indépendant de la capacité, l'écart se creuserait à l'échelle ; ici, il change de signe.

Hypothèse (testée par l'Approche 3, chapitre 13.4) : à faible capacité, la pondération asymétrique de l'attention capte un signal que la somme ignore ; à plus forte capacité, LocalSum absorbe l'hétérogénéité via ses MLPs per-(class, port) et le budget de paramètres supplémentaire de GATv2 se traduit en bruit. MultiHeadQKV (chapitre 13.4) atteint la même best-eval que LocalSum sur Small avec un mécanisme d'attention sans softmax, ce qui isole l'effet softmax-saturating de l'effet capacité brute.

L'Approche 3 (MultiHeadQKV, chapitre 13) apporte un test naturel à cette hypothèse : si une attention linéaire sans softmax atteint la même best-eval que LocalSum sur Small — ce qui se vérifie effectivement au chapitre 13.4 — alors l'écart observé ici avec GATv2 est plutôt attribuable à l'architecture softmax-saturating qu'à un effet brut de capacité.

11.5. Coût et reproductibilité

Gate 6 (perf). Wall-time médian sur forward et forward+backward, mesuré sur 100 appels après 20 itérations de chauffe, avec hyper-paramètres fixés (`in_array_size=4` , `out_size=4` , `hidden=[4]` , seed 64) et JIT compilé via `nnx.jit` :

Substrate	n_addr	LocalSum fwd (ms)	GATv2 fwd (ms)	LocalSum fwd+bwd (ms)	GATv2 fwd+bwd (ms)	overhead fwd	overhead fwd+bwd
LinearSystem	4	1.68	2.85	1.97	3.47	×1.70	×1.76
ieee118	118	7.18	14.24	8.98	17.85	×1.98	×1.99
ieee300	300	7.13	14.16	8.93	17.78	×1.99	×1.99

Le surcoût de GATv2 par rapport à LocalSum est de ×1,70 sur LinearSystem (graphe de petite taille, le dispatch domine), et de ×1,98 à ×1,99 sur ieee118 et ieee300 (régime kernel-bound, deux MLP au lieu d'un). Toutes les valeurs sont en dessous du plafond de ×3 fixé par la section 5 du backlog.

Gate 7 (consistency). L'empreinte SHA-256 du tenseur de sortie de GATv2 (seed 64, contexte IEEE-14 AC-LF) est enregistrée dans `benchmarks/results/01_gatv2/consistency_gatv2.json` , préfixe `21647f27...` . Deux exécutions successives produisent la même empreinte au bit près.

Le solveur AC-LF de pypowsybl n'étant pas bit-déterministe entre processus (l'ordre des threads du solveur sparse introduit une variation float32 qui se propage jusqu'au contexte H2MG), le contexte IEEE-14 est sérialisé une fois lors du premier run, puis rechargé lors des re-runs. Le blob mis en cache constitue l'état de référence figé de la gate. La propriété testée est donc la reproductibilité isolée de GATv2, pas celle de pypowsybl.

11.6. Infrastructure pour les approches suivantes

L'infrastructure pre-cache loader établie au chapitre 10.7 reste partagée par toutes les approches d'attention. Tous les bancs comparent les variantes sur exactement les mêmes données d'entraînement, ce qui élimine le drift float comme facteur confondant entre runs.

Verrouillage du template (`AGENTS.md` , section « Reference implementation: GATv2 »). Le pattern à six fichiers que les Items 2 à 4 répliquent :

- `src/energnn/model/coupler/message_passing/message_passing_function.py` — définition de classe avec docstring math-first, arbres de MLP per-(class, port), discipline gather / scatter ;
- `tests/model/unit/test_<variant>_message_passing.py` — tests unitaires couvrant construction d'arbre, formes de têtes, rétro-compatibilité d'un optional flag, masquage non-fictif, invariants softmax / aggregation, équivariance par permutation, propagation du gradient, sûreté `vmap + jit` , graphe vide, indices hors borne ;
- `tests/model/integration/test_coupler.py` — cas RecurrentCoupler (default + optional flag) ;
- `benchmarks/0X_<variant>/consistency_<variant>.py` — contrôle SHA-256 de reproductibilité sur ieee14 ;
- `benchmarks/0X_<variant>/perf_<variant>_vs_localsum.py` — chronométrage forward et forward+backward médian sur LinearSystem, ieee118 et ieee300 ;
- `docs/tutorials/0X_<variant>.ipynb` — notebook exécutable en deux parties (LinearSystem + IEEE-9/14), chaque cellule de code étant précédée d'une cellule markdown explicative.

L'Item 2 (GlobalAggregation) a été traité en premier en raison de son coût d'implémentation le plus faible. Les Items 3 (MultiHeadQKV), 4 (Performer) et 5 (VirtualAddressRecurrentCoupler) réutilisent la même infrastructure. Les sept portes conservent le même périmètre.

Les chiffres sont issus des fichiers JSON sous `benchmarks/results/00_baseline/` (baseline LocalSum) et `benchmarks/results/01_gatv2/` (Approche 1: `baseline_gatv2_linearsystem.json` , `baseline_gatv2_ac_load_flow.json` , `perf_gatv2_vs_localsum.json` , `consistency_gatv2.json`). L'ensemble des runs a été effectué sur le serveur GPU de La Javaness.

12. Approche 2 — GlobalAggregation (Item 2 du backlog d'attention)

L'Item 2 (`GlobalAggregationMessagePassingFunction`) est livré sur la même feature branch `gatv2-message-passing` . La vérification s'exécute sur le serveur GPU de La Javaness (CUDA, JAX 0.9.0). Ce chapitre regroupe le périmètre d'implémentation, la formulation, les résultats expérimentaux et le compromis caractéristique de l'Approche 2.

12.1. Périmètre d'implémentation et grille de validation

`GlobalAggregationMessagePassingFunction` remplit un stub vide du code source amont, dans le fichier `src/energnn/model/coupler/message_passing/message_passing_function.py` , selon le périmètre v1 de la section 3.2 du backlog : forme `mean` avec dénominateur corrigé. Les sept portes sont exécutées séquentiellement :

Gate	Contenu	Substrat	Statut
1 Unit	13 tests : dimensions du MLP head, invariance par broadcast, dénominateur corrigé (masque 3/10 → mean 1.0), correction de la moyenne vs identité analytique du MLP, invariance par permutation (plus forte que l'équivariance), sûreté <code>vmap + jit</code> , validation du XOR seed / rngs	—	pass (13/13)
2 Static	<code>black</code> , <code>flake8</code> , <code>mypy</code>	—	clean (aucune nouvelle violation)
3 LinearSystem	RecurrentCoupler + GlobalAggregation, Tiny / Small × 3 seeds, entraînement et évaluation	LinearSystem	converge (best-eval Tiny 6.32e-01, Small 4.37e-01)
4 Integration	5 tests dans <code>tests/model/integration/test_coupler.py</code> (un nouveau cas GlobalAgg)	LinearSystem	pass (5/5)
5 IEEE supervised	Entraînement et évaluation Tiny / Small × 3 seeds	ieee9, ieee14	converge (best ieee14 Small 1.85e-02)
6 Perf	Forward et forward+backward médians, 20 warm-up + 100 mesures, JIT	LinearSystem, ieee118, ieee300	plus rapide que LocalSum, overhead ×0.09 à ×0.44
7 Consistency	Empreinte SHA-256 du forward, re-run check	ieee14	bit-identique (préfixe <code>6c2d2c1a...</code>)

12.2. Formulation et dénominateur corrigé

La section 3.2 du backlog pose la formulation suivante :

$$\psi_{\theta}(h, x)_a = \frac{1}{|\mathcal{A}_x|} \sum_{a' \in \mathcal{A}_x} \psi_{\theta}(h_{a'})$$

Couche à vecteur de contexte : la sortie pour chaque adresse réceptrice est le même résumé global. La spécification d'origine utilise le dénominateur $|\mathcal{A}_x|$, c'est-à-dire le nombre total d'adresses, padding fictif compris. Le backlog signale dans ses « Scientific concerns » que le padding dilue le signal après collation et propose un dénominateur corrigé. L'implémentation suit la résolution proposée :

$$\psi_{\theta}(h, x)_a = \sigma \left(\frac{1}{|\mathcal{A}_x^{\text{real}}| + \varepsilon} \sum_{a' \in \mathcal{A}_x^{\text{real}}} \xi_{\theta}(h_{a'}) \right)$$

où ξ_{θ} est un unique valeur MLP par adresse, $\mathcal{A}_x^{\text{real}}$ est l'ensemble des adresses non fictives, $\varepsilon = 10^{-9}$ garde contre le cas dégénéré (ensemble réel vide), et σ est l' `outer_activation` . Implémentation :

```
values = self.value_mlp(coordinates) * mask          # (n_addr, out_size)
total = jnp.sum(values, axis=0)                      # (out_size,)
n_real = jnp.sum(graph.non_fictitious_addresses)    # scalar
mean = total / (n_real + self.eps)                  # (out_size,)
out = jnp.broadcast_to(mean, (n_addr, out_size)) * mask
return self.outer_activation(out), {}
```

Aucune factorisation per-(class, port) — la formulation de la section 3.2 opère au niveau adresse —, aucun softmax (mean linéaire, sans `exp`), aucune stabilisation par segment-max nécessaire. L'**invariance** par permutation (plus forte que l'équivariance de GATv2 et de LocalSum) découle directement de la symétrie de la moyenne en ses arguments.

Conformément à la résolution proposée du backlog, la version v1 retient uniquement la forme `mean` ; les variantes multi-tête, pondérée par attention et avec réducteurs `min / max / sum` sont reportées en v2.

12.3. Résultats Gate 5 (IEEE supervisé AC LF)

Le setup reprend exactement la Gate 5 de l'Approche 1: `ACLoadFlowProblemLoader` avec `perturbation_scale=0.1`, `dataset_size=32`, `batch_size=4`, `val_dataset_size=16`, seeds `[0, 1, 2]`, optimiseur `optax.adam(1e-3)`. Les configurations Tiny et Small sont identiques. Les deux message functions s'exécutent sur le même pipeline sur le serveur GPU de La Javaness.

Network	Size	LocalSum n_params	GlobalAgg n_params	LocalSum best-eval	GlobalAgg best-eval	Δ best-eval	LocalSum train (s)	GlobalAgg train (s)
ieee9	Tiny	1587	719	1.561e-01	2.469e-01	+58.2%	230.3	245.6
ieee9	Small	15863	6879	5.075e-03	2.662e-02	+424.4%	573.0	386.3
ieee14	Tiny	1587	719	8.668e-02	1.482e-01	+70.9%	416.1	309.9
ieee14	Small	15863	6879	4.564e-03	1.846e-02	+304.5%	474.9	504.1

Per-seed best-eval (GlobalAggregation):

Network	Size	seed 0	seed 1	seed 2
ieee9	Tiny	2.655e-01	2.469e-01	2.197e-01
ieee9	Small	3.283e-02	1.602e-02	2.662e-02
ieee14	Tiny	1.401e-01	1.534e-01	1.482e-01
ieee14	Small	1.846e-02	2.114e-02	1.728e-02

12.4. Observation sur la portée des messages

L'Approche 2 produit un profil opposé à l'Approche 1 : ~55 % de paramètres en moins, forward x0,09 plus rapide, best-eval x4 à x5 plus haute. Dans l'Approche 1, GATv2 compte plus de paramètres que LocalSum mais la best-eval reste partagée (Tiny en faveur de GATv2, Small en faveur de LocalSum) — l'écart change de signe avec la capacité, sans gain net de l'attention à 3 seeds.

GlobalAggregation présente un profil différent : environ 55 % de paramètres en moins que LocalSum, un forward 10x plus rapide à la Gate 6, mais une best-eval 1,5 à 5x plus mauvaise sur toutes les paires (réseau, taille). La cause est structurelle :

- L'agrégation per-(class, port) de LocalSum préserve la structure locale de Kirchhoff, nécessaire à la précision sur AC LF. Chaque adresse réceptrice reçoit une information propre, dérivée de ses voisins.
- GlobalAggregation supprime entièrement la structure locale : toutes les adresses reçoivent la même moyenne globale. Le goulet d'étranglement informationnel est fort, et il devient impossible de distinguer un bus situé dans une zone dense d'un bus en bout de réseau.
- Conséquence : sur AC LF, tâche sensible à la topologie, la précision de GlobalAgg est inférieure (x4 à x5), malgré un coût d'exécution réduit.

Ce résultat est exactement celui qu'annonce la section 3.2 du backlog (« Our reading ») : GlobalAggregation construit une couche à vecteur de contexte, où la sortie pour chaque adresse réceptrice est le même résumé global de l'état par adresse ; intégrée à un `RecurrentCoupler` , et combinée à d'autres messages per-address via ϕ , elle ouvre à chaque adresse une fenêtre sur l'ensemble du graphe. C'est la brique élémentaire utilisée par `VirtualAddressRecurrentCoupler` (Approche 5, chapitre 15). Citation originale :

« Couche de contexte global : la sortie pour chaque adresse réceptrice est le même résumé global de l'état per-adresse. Au sein d'un `RecurrentCoupler`, lorsqu'elle est combinée avec d'autres messages per-adresse via ϕ , elle donne à chaque adresse une fenêtre sur l'ensemble du graphe. Brique destinée à entrer dans `VirtualAddressRecurrentCoupler`. »

GlobalAggregation n'est donc **pas un substitut** à l'agrégation par somme. Son rôle architectural est celui de brique pour l'Item 5 (`VirtualAddressRecurrentCoupler`). Dans ce coupler, GlobalAggregation fournit un contexte global par adresse, combiné aux messages per-address (type `LocalSum` ou `GATv2`) via ϕ . Chaque adresse reçoit alors deux signaux distincts : les messages per-(class, port) (signal topologique local) et le résumé de GlobalAggregation (statistique agrégée sur toutes les adresses). La vérification empirique de cette composition est traitée au chapitre 15 (Approche 5, configuration v1 avec `LocalSum` enveloppé).

Les chiffres autonomes du présent chapitre suivent la propriété attendue par la spécification du backlog. GlobalAggregation a moins de paramètres que `LocalSum` (environ 55 % de moins) et un forward d'un facteur 10 plus rapide ; en contrepartie, le broadcast sur l'ensemble du graphe crée un goulet d'étranglement qui dégrade la précision sur AC LF.

12.5. Coût et reproductibilité

Gate 6 (perf). Wall-time médian sur forward et forward+backward, mesuré sur 100 appels après 20 itérations de chauffe, JIT compilé via `nnx.jit` :

Substrate	n_addr	LocalSum fwd (ms)	GlobalAgg fwd (ms)	LocalSum fwd+bwd (ms)	GlobalAgg fwd+bwd (ms)	overhead fwd	overhead fwd+bwd
LinearSystem	4	2.07	0.81	2.18	0.97	×0.39	×0.44
ieee118	118	9.56	0.86	12.21	1.16	×0.09	×0.09
ieee300	300	9.04	0.88	11.53	1.06	×0.10	×0.09

GlobalAggregation est 10× plus rapide que `LocalSum` sur les substrats AC-LF (ieee118, ieee300) et 2,5× plus rapide sur `LinearSystem`. Le forward se réduit à un unique MLP par adresse, suivi d'une moyenne et d'un broadcast, là où `LocalSum` exécute un arbre de MLPs per-(class, port) suivi d'un `scatter_add` par port. La tendance est opposée à celle de l'Approche 1, où le forward de `GATv2` est 2× plus lent que celui de `LocalSum`.

Gate 7 (consistency). L'empreinte SHA-256 du tenseur de sortie (seed 64, contexte IEEE-14 AC-LF) est enregistrée dans `benchmarks/results/02_global_aggregation/consistency_global_aggregation.json`, préfixe `6c2d2c1a...`. Deux exécutions successives produisent la même empreinte au bit près. Le préfixe diffère de celui de l'Approche 1 (`21647f27...`) en raison d'une message function différente. Le même schéma de cache de contexte sérialisé est utilisé pour isoler le non-déterminisme du solveur `pypowsybl`.

12.6. Infrastructure et Items 3-4

Arborescence : scripts dans `benchmarks/02_global_aggregation/`, résultats dans `benchmarks/results/02_global_aggregation/`, notebook `docs/tutorials/02_global_aggregation.ipynb` au format deux parties (`LinearSystem` + IEEE-9/14). Le pattern de l'Approche 1 est conservé : six fichiers par approche.

L'infrastructure pre-cache loader établie au chapitre 10.7 reste partagée. Les Items 3 (`MultiHeadQKV`) et 4 (`Performer`) la réutilisent tels quels, avec le même template et le même workflow strict sur le serveur GPU de La Javaness.

L'Item 3 (`MultiHeadQKVMessagePassingFunction`, section 3.3 du backlog) est traité ensuite (chapitre 13), puis l'Item 4 (`PerformerMessagePassingFunction`, section 3.4 ; chapitre 14). L'Item 5 (`VirtualAddressRecurrentCoupler`, section 3.5) est traité au chapitre 15 sur la branche feature dédiée `virtual-address-coupler`.

13. Approche 3 — MultiHeadQKV (Item 3 du backlog d'attention)

L'Item 3 (`MultiHeadQKVMessagePassingFunction`) est livré sur la même feature branch `gatv2-message-passing` . La vérification s'exécute sur le serveur GPU de La Javaness (CUDA, JAX 0.9.0). Ce chapitre regroupe le périmètre d'implémentation, la formulation Q/K/V avec score bilinéaire et facteur d'échelle, les résultats inter-approches et les observations sur la projection et la mise à l'échelle.

13.1. Périmètre d'implémentation et grille de validation

`MultiHeadQKVMessagePassingFunction` remplit un stub vide du code source amont, dans le fichier `src/energnn/model/coupler/message_passing/message_passing_function.py` (lignes 447 à 465 avant l'Approche 3), selon le périmètre v1 de la section 3.3 du backlog : Q/K/V single-head bilinéaire avec mise à l'échelle en $1/\sqrt{d_{QK}}$. Les sept portes sont exécutées séquentiellement :

Gate	Contenu	Substrat	Statut
1 Unit	15 tests : initialisation de l'arbre de paramètres, dimensions de sortie de Q/K/V, bascule <code>scale_scores</code> et vérification numérique du facteur $\sqrt{d_{QK}}$, masquage des récepteurs fictifs, équivariance par permutation, sûreté <code>vmap + jit</code> , propagation du gradient à travers Q/K/V, sortie nulle sur graphe vide, validation du XOR seed / rngs	—	pass (15/15)
2 Static	<code>black</code> , <code>flake8</code> , <code>mypy</code>	—	clean (aucune nouvelle violation)
3 LinearSystem	RecurrentCoupler + MultiHeadQKV, Tiny / Small × 3 seeds, entraînement et évaluation	LinearSystem	converge (best-eval Tiny 4.35e-01, Small 3.77e-01)
4 Integration	6 tests dans <code>tests/model/integration/test_coupler.py</code> (un nouveau cas MultiHeadQKV)	LinearSystem	pass (6/6)
5 IEEE supervised	Entraînement et évaluation Tiny / Small × 3 seeds	ieee9, ieee14	au niveau de LocalSum sur Small (best ieee14 Small 4.635e-03)
6 Perf	Forward et forward+backward médians, 20 warm-up + 100 mesures, JIT	LinearSystem, ieee118, ieee300	overhead forward ×2.07 , stable sur les trois substrats
7 Consistency	Empreinte SHA-256 du forward, re-run check	ieee14	bit-identique (préfixe <code>6b976412...</code>)

13.2. Formulation Q/K/V et facteur d'échelle $1/\sqrt{d_{QK}}$

La section 3.3 du backlog pose la formulation single-head suivante :

$$Q_a = Q_\theta(h_a) \in \mathbb{R}^{d_{QK}}, \quad K_{c,e,o} = K_\theta^{c,o}(h_e, x_e), \quad V_{c,e,o} = V_\theta^{c,o}(h_e, x_e)$$
$$\psi_\theta(h, x)_a = \sigma \left(\sum_{(c,e,o) \in \mathcal{N}_x(a)} \frac{K_{c,e,o}^\top Q_a}{\sqrt{d_{QK}}} V_{c,e,o} \right)$$

La factorisation per-(class, port) sur K et V reprend le pattern de l'Approche 1 (GATv2) et du baseline LocalSum : l'entrée est la concaténation des features de l'hyper-arête (s'il y en a) et des coordonnées rapatriées via le port, K projette dans $\mathbb{R}^{d_{QK}}$ et V dans $\mathbb{R}^{d_V}$. Q est un unique MLP par adresse (entrée h_a , sortie $\mathbb{R}^{d_{QK}}$), conforme à la spécification $Q_a = Q_\theta(h_a)$.

Motivation du facteur $1/\sqrt{d_{QK}}$. La spécification du backlog utilise $K^\top Q$ brut. L'implémentation s'en écarte pour adopter la convention standard de Vaswani et al. 2017 : diviser par $\sqrt{d_{QK}}$ pour maintenir la variance du score à $\mathcal{O}(1)$ lorsque K et Q sont i.i.d. de variance unitaire. Formellement, si $K, Q \sim \mathcal{N}(0, I_{d_{QK}})$, alors $\text{Var}(K^\top Q) = d_{QK}$ (somme de d_{QK} produits i.i.d. $K_i Q_i$ de variance 1). Le score croît donc en $\sqrt{d_{QK}}$ en amplitude, l'activation extérieure (`tanh`) sature lorsque d_{QK} devient grand, et le gradient s'évanouit.

Ce pattern rejoint celui du dénominateur corrigé de l'Approche 2 : une déviation de la spécification littérale, motivée par la stabilité, et adoptée comme résolution proposée. Le flag `scale_scores=True` est actif par défaut et reste configurable pour les ablations ultérieures (« with vs without scaling »).

Multi-tête reporté en v2. La version single-head est livrée en premier ; la version multi-tête se construit par stacking $d_{QK} \rightarrow H \times d_{QK}/H$ et bouclage de l'équation de spécification.

Implémentation :

```
q_per_address = self.query_mlp(coordinates)                # (n_addr, d_qk)
scale = 1.0 / sqrt(d_qk) if scale_scores else 1.0

for (class, port) in graph:
    masked_input = concat(features, port-gathered coords) * non_fict_mask
    K = k_mlp(masked_input) * non_fict_mask                # (n_edge, d_qk)
    V = v_mlp(masked_input) * non_fict_mask                # (n_edge, out_size)
    Q_at_rcv = gather(q_per_address, port_array)           # (n_edge, d_qk)
    score = sum(K * Q_at_rcv, axis=-1, keepdims=True) * scale # (n_edge, 1)
    weighted_V = V * score * non_fict_mask                 # (n_edge, out_size)
    accumulator = scatter_add(accumulator, weighted_V, port_array)

return self.outer_activation(accumulator), {}
```

Le forward s'effectue en une seule passe, à comparer aux deux passes de GATv2 pour la soustraction du segment-max du softmax. L'équivariance par permutation est conservée, le `scatter_add` étant commutatif sur l'axe source.

13.3. Résultats Gate 5 (IEEE supervisé AC LF)

Le setup reprend exactement celui des Approches 1 et 2 : `ACLoadFlowProblemLoader` avec `perturbation_scale=0.1`, `dataset_size=32`, `batch_size=4`, `val_dataset_size=16`, seeds `[0, 1, 2]`, optimiseur `optax.adam(1e-3)`. Configurations Tiny et Small identiques. Toutes les message fonctions s'exécutent sur le même pipeline.

Réseau	Config	LocalSum n_params	MultiHeadQKV n_params	LocalSum best-eval	MultiHeadQKV best-eval	Δ best- eval	LocalSum train (s)	MultiHeadQKV train (s)
ieee9	Tiny	1587	3403	1.561e-01	2.043e-01	+30.9%	230.3	286.1
ieee9	Small	15863	25407	5.075e-03	4.920e-03	-3.1%	573.0	526.7
ieee14	Tiny	1587	3403	8.668e-02	1.377e-01	+58.9%	416.1	366.8
ieee14	Small	15863	25407	4.564e-03	4.635e-03	+1.6%	474.9	672.0

Best-eval par seed (MultiHeadQKV) :

Réseau	Config	seed 0	seed 1	seed 2
ieee9	Tiny	2.043e-01	1.329e-01	2.533e-01
ieee9	Small	2.456e-03	8.241e-03	4.920e-03
ieee14	Tiny	1.377e-01	1.143e-01	1.556e-01
ieee14	Small	4.635e-03	4.005e-03	9.770e-03

13.4. Comparaison entre approches

L'Approche 3 fait apparaître un schéma différent des deux précédentes (best-eval MultiHeadQKV à <2 % de LocalSum sur Small contre +75 % pour GATv2 et +300 à 500 % pour GlobalAggregation). Le tableau ci-dessous compare les best-eval médianes sur AC LF Small :

Mécanisme	n_params (Small)	ieee9 best-eval	ieee14 best-eval	Gate 6 overhead fwd
LocalSum (baseline)	15863	5.075e-03	4.564e-03	×1.00 (ref)
GATv2 (Approche 1)	25289	5.837e-03	7.995e-03	×1.98
GlobalAggregation (Approche 2)	6879	2.662e-02	1.846e-02	×0.09
MultiHeadQKV (Approche 3)	25407	4.920e-03	4.635e-03	×2.07

Parmi les variantes d'attention testées, MultiHeadQKV est la seule dont la best-eval médiane reste à moins de 2 % de LocalSum sur ieee9 et ieee14 Small. Elle est meilleure que GATv2 de 15 à 42 %, et que GlobalAggregation de 75 à 81 %. Deux écarts par rapport à GATv2 expliquent ce niveau : projections Q/K/V explicites et facteur d'échelle $1/\sqrt{d_{QK}}$. Détail ci-dessous.

- **Projections Q/K/V explicites** à la place du score MLP scalaire de GATv2. Q et K projettent dans $\mathbb{R}^{d_{QK}}$, V dans $\mathbb{R}^{d_V}$. Le score bilinéaire $K^\top Q$ est l'attention par produit scalaire canonique : il utilise d_{QK} degrés de liberté par paire (Q, K) contre 1 scalaire pour le score MLP de GATv2.
- **Mise à l'échelle en $1/\sqrt{d_{QK}}$** qui maintient la variance du score à $\mathcal{O}(1)$ sous l'hypothèse Q, K i.i.d. unité, et limite la saturation de l'activation extérieure pour la valeur par défaut $d_{QK} = 8$. Un test unitaire vérifie l'identité $\text{out}_{\text{off}} = \sqrt{d_{QK}} \cdot \text{out}_{\text{on}}$.
- **Pas de softmax** : l'attention linéaire avec facteur d'échelle suffit à rester compétitive sur des graphes de réseaux électriques (degré typique 2 à 4, l'attention sélective par softmax n'apporte pas d'avantage déterminant). Cela évite à la fois le coût des deux passes et l'évanouissement du gradient lié à la saturation du softmax.

Coût : environ 60 % de paramètres en plus par rapport à LocalSum (25 407 contre 15 863) et un forward 2× plus lent (×2,07 stable sur les trois substrats). Le gain de best-eval reste modeste — de l'ordre de 3 % sur ieee9 Small pour ce surcoût en paramètres et en compute.

Rôle architectural : parmi les Approches 1 à 3, MultiHeadQKV est celle dont la best-eval se rapproche le plus de LocalSum sur ieee9 et ieee14 Small. L'Item 5 (`VirtualAddressRecurrentCoupler` , chapitre 15) compose plusieurs message functions via un état virtuel partagé et un ϕ étendu.

13.5. Coût et reproductibilité

Gate 6 (perf). Wall-time médian sur forward et forward+backward :

Substrat	n_addr	LocalSum fwd (ms)	MultiHeadQKV fwd (ms)	LocalSum fwd+bwd (ms)	MultiHeadQKV fwd+bwd (ms)	overhead fwd	overhead fwd+bwd
LinearSystem	4	1.83	3.45	2.08	4.31	×1.88	×2.07
ieee118	118	9.73	20.12	11.83	25.58	×2.07	×2.16
ieee300	300	9.27	19.20	12.56	25.59	×2.07	×2.04

L'overhead forward de MultiHeadQKV est **stable à ×2.07** sur les trois substrats, légèrement supérieur à celui de l'Approche 1 (×1,98) en raison d'une projection Q supplémentaire par adresse et de projections K et V par arête (au lieu du couple score + valeur de GATv2). Le coût est prévisible et indépendant de l'échelle.

Gate 7 (consistency). L'empreinte SHA-256 du tenseur de sortie (seed 64, contexte IEEE-14 AC-LF) est enregistrée dans `benchmarks/results/03_multi_head_qkv/consistency_multi_head_qkv.json` , préfixe `6b976412...` . Deux re-runs successifs sont bit-identiques. Le préfixe diffère de celui de l'Approche 1 (`21647f27...`) et de l'Approche 2 (`6c2d2c1a...`) en raison d'une message function différente. Le même schéma de cache de contexte sérialisé est utilisé pour isoler le non-déterminisme du solveur `pypowsybl`.

13.6. Infrastructure et Items 4-5

Arborescence : scripts dans `benchmarks/03_multi_head_qkv/` , résultats dans `benchmarks/results/03_multi_head_qkv/` , notebook `docs/tutorials/03_multi_head_qkv.ipynb` au format deux parties (LinearSystem + IEEE-9/14). Le pattern des Approches 1 et 2 est conservé : six fichiers par approche.

L'infrastructure pre-cache loader établie au chapitre 10.7 reste partagée par les trois approches.

L'Item 4 (PerformerMessagePassingFunction , section 3.4 du backlog) est traité au chapitre 14 (attention linéaire all-to-all sans softmax, single-head v1). L'Item 5 (VirtualAddressRecurrentCoupler , section 3.5) est traité au chapitre 15.

14. Approche 4 — Performer (Item 4 du backlog d'attention)

L'Item 4 (PerformerMessagePassingFunction) est livré conformément à la spec du backlog section 3.4 (v1 single-head, sans softmax, sans noyau à features aléatoires). La vérification s'exécute sur le serveur GPU de La Javaness (CUDA, JAX 0.9.0). Ce chapitre regroupe le périmètre d'implémentation, la formulation linéaire all-to-all et sa reformulation kernel-trick, les résultats expérimentaux et leur interprétation par rapport à l'Approche 2 (GlobalAggregation).

14.1. Périmètre d'implémentation et grille de validation

PerformerMessagePassingFunction remplit un stub vide du code source amont, dans src/energnn/model/coupler/message_passing/message_passing_function.py , conformément à la section 3.4 du backlog d'attention. Le périmètre v1 est restreint : une seule tête, pas de softmax, pas de noyau à features aléatoires (Choromanski et al. 2021 reste une extension future). Sept portes de validation sont exécutées séquentiellement :

Gate	Contenu	Substrat	État
1 Unit	15 tests : dimensions Q/K/V, forme de la sortie, parité Form A (somme naïve per-récepteur) vs Form B (kernel-trick utilisé dans l'implémentation), bascule scale_scores + vérification numérique du facteur $\sqrt{d_{QK}}$, masque des adresses fictives, équivariance par permutation, sûreté vmap + jit , flux de gradient à travers Q/K/V, cas de graphe entièrement fictif	—	pass (15/15)
2 Static	black , flake8	—	clean (pas de nouvelle violation introduite)
3 LinearSystem	RecurrentCoupler + Performer, Tiny/Small × 3 seeds, train + eval	LinearSystem	converge (best-eval Tiny 7.24e-01, Small 4.27e-01)
4 Integration	7 tests dans tests/model/integration/test_coupler.py (un nouveau cas Performer)	LinearSystem	pass (7/7)
5 IEEE supervisé	Train + eval Tiny/Small × 3 seeds	ieee9, ieee14	Performer en retrait de LocalSum 4-5x sur Small (best ieee14 Small 2,26e-02)
6 Perf	Médiane forward / forward+backward, 20 warm-up + 100 mesures, JIT	LinearSystem, ieee118, ieee300	forward quasi-constant ≈1,7 ms ; rapport ×0,79 / ×0,21 / ×0,21 vs LocalSum
7 Consistency	SHA-256 du tenseur forward, vérification de re-run	ieee14	bit-identique (413b6662...)
8 Point figé réseau français	Point figé du réseau	—	différé (snapshot non encore disponible)

14.2. Formulation linéaire all-to-all et reformulation kernel-trick

La section 3.4 du backlog pose la formulation single-head suivante :

$$Q_a = Q_\theta(h_a) \in \mathbb{R}^{d_{QK}}, \quad K_{a'} = K_\theta(h_{a'}) \in \mathbb{R}^{d_{QK}}, \quad V_{a'} = V_\theta(h_{a'}) \in \mathbb{R}^{d_V}$$

$$\psi_\theta(h, x)_a = \sigma \left(\frac{1}{\sqrt{d_{QK}}} \sum_{a' \in \mathcal{A}_x^{\text{real}}} (K_{a'}^\top Q_a) V_{a'} \right)$$

avec $\mathcal{A}_x^{\text{real}}$ l'ensemble des adresses non fictives (les adresses paddées sont masquées avant la somme) et σ une activation appliquée après agrégation. Trois MLP au niveau adresse — $Q_\theta, K_\theta, V_\theta$ — agissent sur les coordonnées `coordinates`. La connectivité du graphe (`port_dict` des hyper-arêtes) n'est **pas** consultée dans cette couche : l'agrégation parcourt toutes les adresses sans distinction de classe ni de port.

Reformulation kernel-trick. La spec donne explicitement la factorisation suivante :

$$\psi_\theta(h, x)_a = \sigma \left(\frac{1}{\sqrt{d_{QK}}} \left(\sum_{a'} V_{a'} K_{a'}^\top \right) Q_a \right)$$

L'implémentation accumule une seule fois la matrice $M = \sum_{a'} V_{a'} K_{a'}^\top \in \mathbb{R}^{d_V \times d_{QK}}$ puis multiplie par Q_a pour chaque récepteur. Le coût total est $\mathcal{O}(n_{\text{addr}} \cdot d_V \cdot d_{QK})$ au lieu de $\mathcal{O}(n_{\text{addr}}^2 \cdot d_V)$ pour la somme naïve per-récepteur. Avec d_V, d_{QK} petits et fixes, le forward devient quasi-constant en taille de substrat (Gate 6 mesurée à 1,66/1,70 ms sur iee118/300, cf. 14.5).

L'unit test `test_kernel_trick_form_matches_naive_form` vérifie que les deux formes produisent une sortie numériquement identique à 10^{-4} près sur LinearSystem, ce qui garantit l'absence d'erreur de transposition ou d'einsum dans la factorisation.

Facteur d'échelle $1/\sqrt{d_{QK}}$. Le texte du backlog écrit la forme bilinéaire brute $K^\top Q$. L'implémentation suit la convention de Vaswani et al. 2017 et divise le score par $\sqrt{d_{QK}}$ pour maintenir la variance du score en $\mathcal{O}(1)$ lorsque Q, K sont i.i.d. de variance unité. Le flag `scale_scores=True` par défaut, configurable pour ablation contre la spec littérale du backlog. Même rationale qu'à l'Approche 3 (chapitre 13.2).

Pas de softmax, pas de noyau aléatoire. La spec présente la forme sans softmax (ce qui rend la factorisation possible) et sans approximation par features aléatoires. Le nom `PerformerMessagePassingFunction` est conservé pour respecter le choix du backlog amont, mais l'implémentation correspond plus précisément au "Linear Transformer" de Katharopoulos et al. 2020. L'approximation par features aléatoires (FAVOR+, Choromanski et al. 2021) est différée en extension future.

Implémentation :

```
q = self.query_mlp(coordinates)          # (n_addr, d_qk)
k = self.key_mlp(coordinates)            # (n_addr, d_qk)
v = self.value_mlp(coordinates)          # (n_addr, out_size)

mask = jnp.expand_dims(graph.non_fictitious_addresses, -1)
k_masked = k * mask
v_masked = v * mask

m = jnp.einsum("ad,af->df", v_masked, k_masked) # (out_size, d_qk)
scale = 1.0 / jnp.sqrt(jnp.float32(self.d_qk)) if self.scale_scores else 1.0
output = jnp.einsum("df,af->ad", m, q) * scale # (n_addr, out_size)

return self.outer_activation(output), {}
```

Aucun appel à `gather` ni `scatter_add` : le `port_dict` de la connectivité n'est pas consulté. Les opérations sont deux einsums plus un masquage, indépendantes du nombre d'hyper-arêtes.

14.3. Résultats Gate 5 (IEEE supervisé AC LF)

Le setup reprend exactement la Gate 5 des Approches 1-3 : `ACLoadFlowProblemLoader` avec `perturbation_scale=0.1`, `dataset_size=32`, `batch_size=4`, `val_dataset_size=16`, `seeds [0, 1, 2]`, optimiseur `optax.adam(1e-3)`. Configurations Tiny et Small identiques. Les deux message functions s'exécutent sur le même pipeline sur le serveur GPU de La Javaness. Temps total d'entraînement sur les 12 runs : 68 minutes.

Réseau	Taille	LocalSum n_params	Performer n_params	LocalSum best-eval	Performer best-eval	Δ best-eval	LocalSum train (s)	Performer train (s)
ieee9	Tiny	1587	799	1,561e-01	2,039e-01	+30,6%	230,3	183,6
ieee9	Small	15863	7439	5,075e-03	3,067e-02	+504,4%	573,0	356,4
ieee14	Tiny	1587	799	8,668e-02	1,221e-01	+40,9%	416,1	303,6
ieee14	Small	15863	7439	4,564e-03	2,264e-02	+396,1%	474,9	453,5

Performer a environ 47% du nombre de paramètres de LocalSum sur ieee Small (7439 vs 15863) car ses trois MLPs Q/K/V opèrent dans l'espace coordonnée (dimension `latent_dim`) plutôt que dans l'espace concaténé per-(class, port) de LocalSum. La best-eval est en retrait par rapport à LocalSum : $\times 4$ à $\times 6$ plus élevée sur Small, +30 à +41% sur Tiny. Best-eval par seed (Performer) :

Réseau	Taille	seed 0	seed 1	seed 2
ieee9	Tiny	2,039e-01	1,930e-01	2,385e-01
ieee9	Small	3,598e-02	3,067e-02	2,881e-02
ieee14	Tiny	1,221e-01	1,187e-01	1,375e-01
ieee14	Small	2,264e-02	1,903e-02	2,387e-02

14.4. Comparaison entre approches

L'Approche 4 produit un profil voisin de l'Approche 2 (GlobalAggregation) sur AC LF Small et distinct des Approches 1 (GATv2) et 3 (MultiHeadQKV). Comparaison best-eval médiane sur AC LF Small :

Mécanisme	n_params (Small)	ieee9 best-eval	ieee14 best-eval	Gate 6 fwd
LocalSum (baseline)	15863	5,075e-03	4,564e-03	$\times 1,00$ (réf)
GATv2 (Approche 1)	25289	5,837e-03	7,995e-03	$\times 1,98$
GlobalAggregation (Approche 2)	6879	2,662e-02	1,846e-02	$\times 0,09$
MultiHeadQKV (Approche 3)	25407	4,920e-03	4,635e-03	$\times 2,07$
Performer (Approche 4)	7439	3,067e-02	2,264e-02	$\times 0,21$

Performer rejoint GlobalAggregation dans le groupe « peu de paramètres, forward rapide, eval en retrait sur tâche topologique ». Les deux partagent la même propriété architecturale : agrégation globale sur toutes les adresses sans factorisation per-(class, port). Best-eval Performer ieee9 Small (3,07e-02) reste plus élevée que GlobalAggregation (2,66e-02) mais du même ordre. Symétriquement, GATv2 et MultiHeadQKV — qui exploitent la connectivité — restent autour du seuil 5×10^{-3} de LocalSum.

Sur le forward, Performer est $5\times$ plus rapide que LocalSum sur ieee118/300 ($\times 0,21$), conséquence de la complexité kernel-trick $\mathcal{O}(n_{\text{addr}} \cdot d_V \cdot d_{QK})$ avec d_V, d_{QK} petits et fixes. GlobalAggregation est plus rapide encore ($\times 0,09$) avec un coût encore plus simple (un MLP par adresse, puis moyenne). Performer combine donc un forward du même ordre que GlobalAggregation (1,7 ms) avec une best-eval marginalement plus basse (3,07e-02 vs 2,66e-02 sur ieee9 Small) et 8 % de paramètres en plus (7 439 vs 6 879).

Lecture physique : les graphes power-system ont une structure radiale ou maillée (degré 2-4) qui porte un signal exploité par les message functions per-(class, port). Une attention indifférente à cette topologie — Performer comme GlobalAggregation — perd le signal porté par les hyper-arêtes. LocalSum, GATv2 et MultiHeadQKV exploitent ce signal via leurs arbres per-(class, port).

14.5. Coût (Gate 6) et reproductibilité (Gate 7)

Gate 6 perf. Médiane sur 100 appels après 20 warm-up, identique aux Approches 1-3 :

Substrat	n_addr	LocalSum fwd (ms)	Performer fwd (ms)	LocalSum fwd+bwd (ms)	Performer fwd+bwd (ms)	overhead fwd	overhead fwd+bwd
LinearSystem	4	2,17	1,72	2,03	1,96	$\times 0,79$	$\times 0,97$

Substrat	n_addr	LocalSum fwd (ms)	Performer fwd (ms)	LocalSum fwd+bwd (ms)	Performer fwd+bwd (ms)	overhead fwd	overhead fwd+bwd
ieee118	118	7,99	1,66	10,45	1,98	×0,21	×0,19
ieee300	300	8,01	1,70	10,31	2,00	×0,21	×0,19

Le forward Performer est quasi-indépendant de la taille du substrat (~1,7 ms partout, ~2 ms en forward+backward), conséquence directe de la complexité kernel-trick. LocalSum, lui, croît avec le nombre d'hyper-arêtes (coût per-edge MLP dominant), d'où le rapport ×0,21 sur ieee118/300. Sur LinearSystem (n_addr=4), le ratio est proche de 1 : trop peu d'hyper-arêtes pour que LocalSum prenne du retard. Le profil est inverse de celui de l'Approche 3 (MultiHeadQKV, ×2,07 partout) car MultiHeadQKV reste per-edge tandis que Performer est all-to-all factorisé.

Gate 7 consistency. Empreinte SHA-256 du tenseur de sortie (seed 64, contexte ieee14 AC-LF) :

```
413b66628748dfed3dbfac926e1b9a8f4656338ec95a07ef562c292cb488355e
```

Deux exécutions successives sur le serveur GPU de La Javaness reproduisent cette empreinte au bit près. Préfixe différent des Approches précédentes (Approche 1 GATv2 21647f27..., Approche 2 GlobalAggregation 6c2d2c1a..., Approche 3 MultiHeadQKV 6b976412...) — attendu pour une message function différente. Le contexte ieee14 est sérialisé via pickle une fois puis rechargé pour isoler la reproductibilité Performer du non-déterminisme du solveur pypowsybl. Même schéma de cache que les Approches 1-3.

15. Approche 5 — VirtualAddressRecurrentCoupler (Item 5 du backlog d'attention)

L'Item 5 (VirtualAddressRecurrentCoupler) est livré sur la branche feature dédiée virtual-address-coupler , distincte de la branche rte-delivery qui porte les Approches 1 à 4. La séparation reflète la nature structurellement différente de l'item : il s'agit d'un coupler (couche d'orchestration au-dessus des message functions) et non d'une nouvelle message function. La vérification s'exécute sur le serveur GPU de La Javaness (CUDA, JAX 0.9.0). Ce chapitre regroupe le périmètre d'implémentation, le choix de design encodé pour rendre F_virtual exécutable, et les résultats expérimentaux.

15.1. Périmètre d'implémentation et grille de validation

VirtualAddressRecurrentCoupler remplit un stub partiellement scaffoldé du code source amont, dans src/energnn/model/coupler/message_passing/recurrent_coupler.py (lignes 79 et suivantes, ajoutées par le commit b458a2c de l'amont). Le scaffold contenait deux TODO bloquants à l'exécution : l'injection de h_virtual_address dans les message functions et la définition de F_virtual qui était référencée mais non définie. Les choix retenus sont documentés en section 15.2 et validés par sept portes :

Gate	Contenu	Substrat	État
1 Unit	11 tests dans tests/model/unit/test_virtual_address_recurrent_coupler.py : attributs constructeur, forme et info , finitude, déterminisme par seed, dégénérescence à RecurrentCoupler pour virtual_address_size=0 , pas Euler unique, exclusion des adresses fictives de la moyenne du virtual, plusieurs message functions, sûreté vmap + jit , flux de gradient à travers phi_virtual , évolution déterministe de l'état virtuel	—	pass (11/11)
2 Static	black , flake8	—	clean (pas de nouvelle violation)
3 LinearSystem	VAR+LocalSum, Tiny/Small × 3 seeds, train + eval	LinearSystem	converge (best-eval Tiny 4,45e-01, Small 3,95e-01)
4 Integration	8 tests dans tests/model/integration/test_coupler.py (un nouveau cas VirtualAddressRecurrentCoupler)	LinearSystem	pass (8/8)

Gate	Contenu	Substrat	État
5 IEEE supervisé	Train + eval Tiny/Small × 3 seeds	ieee9, ieee14	VAR+LocalSum reste dans la dispersion inter-seeds de LocalSum sur ieee9 et ieee14 Small (résultat négatif à 3 seeds)
6 Perf	Médiane forward et forward+backward, 20 warm-up + 100 mesures, JIT	LinearSystem, ieee118, ieee300	différé — commit de suivi
7 Consistency	SHA-256 du tenseur forward, vérification de re-run	ieee14	différé — commit de suivi
8 Point figé réseau français	Point figé du réseau	—	différé (snapshot non encore disponible)

15.2. Architecture et choix de design

Le scaffold amont définit la signature suivante (verbatim) : `__init__(phi, message_functions, n_steps, virtual_address_size)` . Deux TODO bloquent l'exécution : l'injection de `h_virtual_address` dans les message functions et la définition de `F_virtual` . Conformément à la convention du backlog section 0 (« nous proposons un ajustement scientifiquement fondé plutôt que de bloquer sur une clarification ; Donon accepte, raffine, ou surcharge la proposition »), les trois questions critiques Q5.1, Q5.2, Q5.3 du backlog reçoivent les réponses suivantes.

Q5.1 — Injection (design (c)). `h_virtual` est concaténé au vecteur de messages avant `phi` , pas avant les message functions. Aucune modification de l'interface `MessagePassingFunction.__call__` n'est nécessaire ; les Approches 1 à 4 sont réutilisées sans migration. La signature de `phi` évolue de `in_size = sum(message_out_sizes)` à `in_size = sum(message_out_sizes) + virtual_address_size` .

Q5.2 — F_virtual (design (alpha)). `F_virtual = phi_virtual(concat(masked_mean(h), h_virtual_old))` , où la moyenne est prise sur les adresses non fictives avec le dénominateur corrigé `sum(non_fictitious_addresses) + eps` , soit le même schéma de masquage que l'Approche 2 (`GlobalAggregationMessagePassingFunction`). `phi_virtual` est une MLP supplémentaire, paramètre du constructeur étendu de l'Item 5 (signature étendue par rapport au scaffold pour rendre `F_virtual` exécutable).

Q5.3 — Adresse virtuelle unique (v1). Un seul vecteur partagé `h_virtual ∈ ℝ^{d_v}` . L'extension à `k` adresses virtuelles est reportée en v2 si v1 produit un gain mesurable.

L'équivariance par permutation des adresses est préservée : la moyenne masquée est invariante, le broadcast à toutes les adresses est invariant, le reste du circuit reprend les opérations équivariantes de `RecurrentCoupler` .

Forward Euler joint. Chaque pas met à jour `h` et `h_virtual` en parallèle, en utilisant `h_old` pour le calcul de la moyenne masquée afin d'éviter une dépendance circulaire dans une seule itération :

```
for _ in range(self.n_steps):
    h_old = h

    # F: per-address residual update with virtual state injected into phi.
    messages = [m(graph=graph, coordinates=h)[0] for m in self.message_functions]
    msg_cat = jnp.concatenate(messages, axis=-1)
    msg_v_broadcast = jnp.broadcast_to(h_virtual, (n_addr, virtual_address_size))
    phi_input = jnp.concatenate([msg_cat, msg_v_broadcast], axis=-1)
    h = h + dt * self.phi(phi_input)

    # F_virtual: virtual state residual update via masked mean of h_old.
    mask = jnp.expand_dims(graph.non_fictitious_addresses, -1)
    h_masked = h_old * mask
```

```
denom = jnp.sum(graph.non_fictitious_addresses) + self.eps
h_mean = jnp.sum(h_masked, axis=0) / denom
virt_input = jnp.concatenate([h_mean, h_virtual], axis=-1)
h_virtual = h_virtual + dt * self.phi_virtual(virt_input)
```

Validation au constructeur. La signature étendue vérifie trois contrats de taille à la construction : `phi.in_size == sum(mf.out_size) + virtual_address_size`, `phi_virtual.in_size == phi.out_size + virtual_address_size`, et `phi_virtual.out_size == virtual_address_size`. Toute violation déclenche un `ValueError` avant la première forward, ce qui évite des messages d'erreur tardifs et opaques dans le code `jit`-compilé.

Dégénérescence à RecurrentCoupler. Avec `virtual_address_size = 0`, le coupler reproduit exactement la sémantique de `RecurrentCoupler` (à l'erreur flottante près). Vérifié par le test unitaire `test_zero_virtual_size_reproduces_recurrent_coupler`.

15.3. Résultats Gate 3 (LinearSystem)

Setup identique aux Gate 3 des Approches 1 à 4. La message function enveloppée est `LocalSumMessagePassingFunction` (référence Donon, Approche 0). `virtual_address_size = latent_dim` (default OQ4 du design proposal). Temps total d'exécution : 13,9 minutes pour 6 runs sur le serveur GPU de La Javaness.

Taille	LocalSum (Approche 0) n_params	VAR+LocalSum n_params	LocalSum best-eval	VAR+LocalSum best-eval	Δ best-eval	LocalSum train (s)	VAR+LocalSum train (s)
Tiny	185	237	4,388e-01	4,446e-01	+1,3%	116,5	93,2
Small	2177	2377	3,877e-01	3,946e-01	+1,8%	227,2	181,1

L'écart médian par rapport à `LocalSum + RecurrentCoupler` reste sous 2 % sur les deux tailles, dans la dispersion à 3 seeds des deux configurations. Le coût ajouté par l'état virtuel sur `LinearSystem` n'apporte pas de signal mesurable sur ce substrat : `LinearSystem` a `n_addr=4` par problème et une topologie réduite, l'apport d'un canal de mémoire global au-dessus d'une agrégation déjà locale est limité.

Détail par seed :

Taille	seed 0	seed 1	seed 2	train (s) médian
Tiny	8,777e-01	3,823e-01	4,446e-01	93,2
Small	4,465e-01	3,583e-01	3,994e-01	181,1

15.4. Résultats Gate 5 (IEEE supervisé AC LF)

Setup identique aux Gate 5 des Approches 1 à 4. La message function enveloppée est `LocalSumMessagePassingFunction` (Approche 0, référence Donon). `virtual_address_size = latent_dim`. Temps total d'exécution : 74,9 minutes pour 12 runs sur le serveur GPU de La Javaness.

Réseau	Taille	LocalSum (Approche 0) n_params	VAR+LocalSum n_params	LocalSum best-eval	VAR+LocalSum best-eval	Δ best-eval	LocalSum train (s)	VAR+LocalSum train (s)
ieee9	Tiny	1587	1639	1,561e-01	1,476e-01	-5,4%	230,3	248,2
ieee9	Small	15863	16063	5,075e-03	6,281e-03	+23,7%	573,0	399,9
ieee14	Tiny	1587	1639	8,668e-02	1,165e-01	+34,4%	416,1	324,8
ieee14	Small	15863	16063	4,564e-03	7,127e-03	+56,1%	474,9	506,1

La médiane `VAR+LocalSum` s'éloigne de la référence `LocalSum` par +23 à +56 % sur trois cellules sur quatre. Avec 3 seeds, l'écart entre les deux mécanismes reste dans la dispersion inter-seeds des `best-eval` : pas de conclusion d'amélioration ni de dégradation à

ce périmètre. Les médianes VAR+LocalSum restent dans la bande d'eval du Groupe A (mécanismes topology-aware), pas dans celle du Groupe B.

Détail par seed sur AC LF Small :

Réseau	Taille	seed 0	seed 1	seed 2	médiane
ieeee9	Small	2,682e-03	8,354e-03	6,281e-03	6,281e-03
ieeee14	Small	3,239e-03	7,127e-03	1,181e-02	7,127e-03

15.5. Comparaison entre approches

Mécanisme	n_params (Small)	ieeee9 best-eval	ieeee14 best-eval	Gate 6 fwd
LocalSum (référence)	15863	5,075e-03	4,564e-03	×1,00 (réf)
GATv2 (Approche 1)	25289	5,837e-03	7,995e-03	×1,98
GlobalAggregation (Approche 2)	6879	2,662e-02	1,846e-02	×0,09
MultiHeadQKV (Approche 3)	25407	4,920e-03	4,635e-03	×2,07
Performer (Approche 4)	7439	2,881e-02	2,013e-02	×0,21
VAR+LocalSum (Approche 5)	16063	6,281e-03	7,127e-03	<i>différée</i>

L'Approche 5 se classe dans la bande du Groupe A (topology-aware) puisque la message function enveloppée est `LocalSumMessagePassingFunction`. Le mécanisme d'état virtuel n'améliore pas la *best-eval* par rapport à `RecurrentCoupler` + `LocalSum` sur ieeee9 et ieeee14 Small au seuil de 3 seeds. La séparation Groupe A / Groupe B reste inchangée : les approches topology-aware (LocalSum, GATv2, MultiHeadQKV, VAR+LocalSum) restent dans la bande 5 à 8×10^{-3} sur ieeee9 Small ; les approches topology-agnostic (GlobalAggregation, Performer) restent autour de 2,7 à $2,9 \times 10^{-2}$.

15.6. Implications et limites du périmètre actuel

L'Approche 5 v1 produit un résultat négatif : le mécanisme d'état virtuel partagé (design (c) injection + design (alpha) F_virtual, vecteur unique), enveloppant la message function LocalSum, n'apporte pas d'amélioration mesurable sur AC LF Small à 3 seeds. Les médianes s'éloignent de la référence par +23 à +56 %, mais l'écart entre les deux mécanismes reste dans la dispersion inter-seeds des best-eval.

La validation Gate 6 (coût) et Gate 7 (consistency) reste à exécuter sur la branche `virtual-address-coupler` en commit de suivi (cf section 15.1, ligne du tableau « Gate 6 » et « Gate 7 »). L'estimation de coût forward est environ `fwd_LocalSum` + `fwd_phi_virtual` ≈ 8 à 10 ms sur ieee118 et ieee300, le ratio par rapport à LocalSum reste donc sous ×1,3 sur les substrats larges et bien sous le seuil ×3 de la Gate 6.

Limites : la validation est conduite uniquement sur la configuration v1 du coupler avec `LocalSumMessagePassingFunction` enveloppée, conformément au design proposé en section 15.2. Les Gate 6 (perf) et Gate 7 (consistency) sont différées en commit de suivi sur la branche `virtual-address-coupler` ; les chiffres sont à reporter ici une fois disponibles. Le périmètre de réseaux est gelé sur ieeee9 et ieeee14 (Gate 5) et LinearSystem (Gate 3), conformément à la convention `feedback_verification_scope` partagée avec les Approches 1 à 4.

Les chiffres sont issus des fichiers JSON sous `benchmarks/results/05_virtual_address/` (`baseline_var_linearsystem.json` et `baseline_var_ac_load_flow.json` une fois Gate 5 terminé). Tous les runs ont été exécutés sur le serveur GPU de La Javaness. Le code source vit sur la branche `virtual-address-coupler` ; la branche `rte-delivery` reste à l'état actuel (Approches 1 à 4 livrées) jusqu'à validation amont du design de l'Item 5.

16. Conclusion et perspectives

Le baseline LocalSum sur LinearSystem et IEEE-9/14 sert de référence pour les cinq Approches d'attention livrées. Chaque Approche est validée avec les mêmes seeds, le même optimizer et les mêmes substrats que LocalSum, ce qui rend les comparaisons directes (sans biais d'environnement). L'Approche 5 (VirtualAddressRecurrentCoupler) clôt la livraison des cinq Items du backlog d'attention (cf. chapitre 15).

16.1. Critères de validation atteints

Trois portes de validation ont fixé les seuils chiffrés au début du travail attention :

- **Gate 3 (LinearSystem)** : Tiny best-eval $\leq 0,44$, Small $\leq 0,39$ (médiane sur 3 seeds). Approches 1-3 respectent ce seuil. Approche 4 (Performer) dépasse le seuil sur Tiny (7,24e-01) et marginalement sur Small (4,27e-01), conséquence de l'indifférence topologique du mécanisme (chapitre 14.3). Voir chapitres 11-13 et 14 pour les chiffres par approche.
- **Gate 5 (IEEE supervisé)** : Small best-eval médiane $iee9 \leq 5,1 \times 10^{-3}$, $iee14 \leq 4,6 \times 10^{-3}$. GATv2 dépasse légèrement le seuil ($5,8$ et $8,0 \times 10^{-3}$, cf. chapitre 11.3). GlobalAggregation reste au-dessus de ce seuil (cf. chapitre 12.4 pour l'interprétation structurelle). MultiHeadQKV atteint le seuil ($4,9$ et $4,6 \times 10^{-3}$, chapitre 13.3). Performer reste au-dessus du seuil ($3,1 \times 10^{-2}$ et $2,3 \times 10^{-2}$, chapitre 14.3), même profil que GlobalAggregation.
- **Gate 6 (perf)** : forward+backward overhead vs LocalSum $\leq 3 \times$ sur iee118 et iee300. Les Approches 1 à 4 sont en dessous : GATv2 et MultiHeadQKV autour de $\times 2$, GlobalAggregation $\times 0,09$, Performer $\times 0,19$. La Gate 6 de l'Approche 5 n'est pas livrée à ce commit (cf. section 15.6). Voir chapitres 11.5, 12.5, 13.5 et 14.5.

L'expérimentation de conversion IEEE sur les 6 réseaux (chapitre 9) reste valide pour les cinq approches : V_mag dans la plage plausible, sorties finies, topologie préservée.

16.2. Bilan d'implémentation

1. ~~Construire ACLoadFlowProblem autour de la sortie pypowsybl-to-energyn~~. **TERMINÉ** — fichier `benchmarks/ac_load_flow_problem.py`.
2. ~~Re-jouer le baseline LocalSum sur IEEE-9/IEEE-14 supervisé et reporter les chiffres dans BASELINES.md~~ : **TERMINÉ** — 12 runs sur le serveur GPU de La Javaness, `BASELINES.md` section 3, résultats au chapitre 10.
3. ~~Notebook de référence docs/tutorials/00_baseline_localsum.ipynb (baseline LocalSum)~~: **TERMINÉ** — démonstrateur exécutable au format deux parties (LinearSystem + IEEE-9/14).
4. ~~Approche 1 (GATv2) à travers les portes 1 à 7 + notebook 01_gatv2.ipynb~~ : **TERMINÉ** — chapitre 11.
5. ~~Approche 2 (GlobalAggregation) à travers les portes 1 à 7 + notebook 02_global_aggregation.ipynb~~ : **TERMINÉ** — chapitre 12.
6. ~~Approche 3 (MultiHeadQKV) à travers les portes 1 à 7 + notebook 03_multi_head_qkv.ipynb~~ : **TERMINÉ** — chapitre 13.
7. ~~Approche 4 (Performer) à travers les portes 1 à 7 + notebook 04_performer.ipynb~~ : **TERMINÉ** — chapitre 14.
8. ~~Approche 5 (VirtualAddressRecurrentCoupler) à travers les portes 1, 2, 3, 4, 5 ; portes 6 et 7 différées en commit de suivi~~: **TERMINÉ (gates 1-5)** — chapitre 15.